Low-Level Software Security for Compiler Developers

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license, visit http://creativecommons.org/licenses/by/4.0/ or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Copyright 2021-2024 Arm Limited open-source-office@arm.com
Copyright 2023 Bill Wendling morbo@google.com
Copyright 2023 Lucian Popescu lucian.popescu187@gmail.com
Copyright 2024 Anders Waldenborg anders@0x63.nu

Version 0-220-g97004b7

Contents

1 Introduction **4**
1.1 Why an open source book? 5

2 Memory vulnerability based attacks **6**
2.1 A bit of background on memory vulnerabilities 6
2.2 Exploitation primitives 7
2.3 Stack buffer overflows 10
2.4 Use After Free (UaF) 13
2.5 Code reuse attacks 16
2.5.1 Return-oriented programming 16
2.5.2 Jump-oriented programming 20
2.5.3 Counterfeit Object-oriented programming 21
2.5.4 Sigreturn-oriented programming 21
2.6 Mitigations against code reuse attacks 21
2.6.1 ASLR 22
2.6.2 Control-flow Integrity (CFI) 22
2.7 Non-control data attacks 33
2.8 Preventing and detecting memory errors 35
2.8.1 Sanitizers 35
2.8.2 Bounds checking 37
2.9 JIT compiler vulnerabilities 39

3 Covert channels and side-channels **43**
3.1 Timing side-channels 43
3.2 Cache side-channels 44
3.2.1 Typical CPU cache architecture 44
3.2.2 Operation of cache side-channels 47
3.2.3 Mitigating cache side-channel attacks 48
3.3 Branch-predictor based side-channels 50
3.3.1 Branch predictors 50
3.3.2 Side-channels through branch predictors 50
3.3.3 Mitigations 51
3.4 Resource contention channels 51
3.5 Channels making use of aliasing in other predictors 51
3.6 Transient execution attacks 52
3.6.1 Transient execution 52
3.6.2 Transient Execution Attacks 54

3.6.3 Mitigations against transient execution attacks 55
3.7 Physical access side-channel attacks 55

4 Supply chain attacks **56**
4.1 History of supply chain attacks 56

5 Compiler introduced security vulnerabilities **58**

6 Underhanded code **62**
6.1 Assignment and equality confusion 62
6.2 goto fail 63
6.3 Trojan Source 64

7 Physical attacks **65**
7.1 Overview 65
7.2 Physical access side-channel attacks 65
7.2.1 How is information leaked? 66
7.2.2 Side channel leakage at instruction level 66
7.2.3 Countermeasures 68
7.3 Fault injection attacks 69
7.3.1 Common forms of Fault injection attacks 69
7.3.2 Countermeasures 70

8 Other security topics relevant for compiler developers **71**

Appendix: contribution guidelines **72**

References **76**

Chapter 1

Introduction

Compilers, assemblers and similar tools generate all the binary code that processors execute. It is no surprise then that these tools play a major role in security analysis and hardening of relevant binary code.

Often the only practical way to protect all binaries with a particular security hardening method is to have the compiler do it. And, with software security becoming more and more important in recent years, it is no surprise to see an ever increasing variety of security hardening features and mitigations against vulnerabilities implemented in compilers. Indeed, compared to a few decades ago, today's compiler developer is much more likely to implement security features than not.

Furthermore, with the ever-expanding range of techniques implemented, it's very hard to gain a basic understanding of all security features implemented in typical compilers.

This poses a practical problem: compiler developers must be able to work on security hardening features, yet it's hard to gain a good, basic understanding of such compiler features.

There are a lot of materials that explain individual vulnerabilities or attack vectors. There are also lots of presentations explaining specific exploits. But there seems to be a limited set of materials that give a structured overview of all vulnerabilities and exploits against which a code generator plays a role in protecting.

This book aims to provide such a structured, broad overview. It does not necessarily go into full details, instead aiming to give a thorough description of all relevant high-level aspects of attacks, vulnerabilities, mitigations, and hardening techniques. For further details, this book provides pointers to materials with more details on specific techniques.

The purpose of this book is to serve as a guide to every compiler developer that needs to learn about software security relevant to compilers. Even though the focus is on compiler developers, we expect that this book will also be useful to people working on other low-level software.

1.1 Why an open source book?

The idea for this book emerged out of a frustration of not finding a good overview on this topic. Kristof Beyls and Georgia Kouveli, both compiler engineers working on security features, wished a book like this would exist. After not finding such a book, they decided to try and write one themselves. They immediately realized that they do not have all necessary expertise themselves to complete such a daunting task. So they decided to try and create this book in an open source style, seeking contributions from many experts.

As you read this, the book remains unfinished. This book may well never be finished, as new vulnerabilities continue to be discovered regularly. Our hope is that developing the book as an open source project will allow for it to continue to evolve and improve. The open source development process of this book increases the likelihood that it remains relevant as new vulnerabilities and mitigations emerge.

Kristof and Georgia, the initial authors, are far from experts on all possible vulnerabilities. So what is the plan to get high quality content to cover all relevant topics? It is two-fold.

First, by studying specific topics, they hope to gain enough knowledge to write up a good summary for this book.

Second, they very much invite and welcome contributions. If you're interested in potentially contributing content, please go to the home location for the open source project at https://github.com/llsoftsec/llsoftsecbook.

As this book is very much a work in progress, you will notice plenty of "TODO" items throughout this book.

This is an example "TODO" item

We think that keeping the TODOs clearly visible helps both readers and potential contributors. Readers get hints of what kind of further content is still missing. Potential contributors get hints for where we need help to further improve this book.

As a reader, you can also contribute to making this book better. We highly encourage feedback, both positive and constructive criticisms. We prefer feedback through either https://github.com/llsoftsec/llsoftsecbook, for example by raising an issue there, or through our Discord server. Interested potential contributors can reach the LLSoftSec community through the same channels.

Add section describing the structure of the rest of the book.

Chapter 2

Memory vulnerability based attacks

2.1 A bit of background on memory vulnerabilities

Memory access errors describe memory accesses that, although permitted by a program, were not intended by the programmer. These types of errors are usually defined (Hicks 2014) by explicitly listing their types, which include:

- buffer overflow
- null pointer dereference
- use after free
- use of uninitialized memory
- illegal free

Memory vulnerabilities are an important class of vulnerabilities that arise due to these types of errors, and they most commonly occur due to programming mistakes when using languages such as C/C++. These languages do not provide mechanisms to protect against memory access errors by default. An attacker can exploit such vulnerabilities to leak sensitive data or overwrite critical memory locations and gain control of the vulnerable program.

Memory vulnerabilities have a long history. The Morris worm in 1988 was the first widely publicized attack exploiting a buffer overflow. Later, in the mid-90s, a few famous write-ups describing buffer overflows appeared (Aleph One 1996). Stack buffer overflows were mitigated with stack canaries and non-executable stacks. The answer was more ingenious ways to bypass these mitigations: code reuse attacks, starting with attacks like return-into-libc (Solar Designer 1997). Code reuse attacks later evolved to Return-Oriented Programming (ROP) (Shacham 2007) and even more complex techniques.

To defend against code reuse attacks, the Address Space Layout Randomization (ASLR) and Control-Flow Integrity (CFI) measures were introduced. This interaction between offensive and defensive security research has been essential

to improving security, and continues to this day. Each newly deployed mitigation results in attempts, often successful, to bypass it, or in alternative, more complex exploitation techniques, and even tools to automate them.

Memory safe (Hicks 2014) languages are designed with prevention of such vulnerabilities in mind and use techniques such as bounds checking and automatic memory management. If these languages promise to eliminate memory vulnerabilities, why are we still discussing this topic?

On the one hand, C and C++ remain very popular languages, particularly in the implementation of low-level software. On the other hand, programs written in memory safe languages can themselves be vulnerable to memory errors as a result of bugs in how they are implemented, e.g. a bug in their compiler. Can we fix the problem by also using memory safe languages for the compiler and runtime implementation? Even if that were as simple as it sounds, unfortunately there are types of programming errors that these languages cannot protect against. For example, a logical error in the implementation of a compiler or runtime for a memory safe language can lead to a memory access error not being detected. We will see examples of such logic errors in compiler optimizations in a later section.

Given the rich history of memory vulnerabilities and mitigations and the active developments in this area, compiler developers are likely to encounter some of these issues over the course of their careers. This chapter aims to serve as an introduction to this area. We start with a discussion of exploitation primitives, which can be useful when analyzing threat models . We then continue with a more detailed discussion of the various types of vulnerabilities, along with their mitigations, presented in a rough chronological order of their appearance, and, therefore, complexity.

Discuss threat models elsewhere in book and refer to that section here #161

2.2 Exploitation primitives

Newcomers to the area of software security may find themselves lost in many blog posts and other publications describing specific memory vulnerabilities and how to exploit them. Two very common, yet unfamiliar to a newcomer, terms that appear in such publications are *read primitive* and *write primitive*. In order to understand memory vulnerabilities and be able to design effective mitigations, it's important to understand what these terms mean, how these primitives could be obtained by an attacker, and how they can be used.

An *exploit primitive* is a mechanism that allows an attacker to perform a specific operation in the memory space of the victim program. This is done by providing specially crafted input to the victim program.

A *write primitive* gives the attacker some level of write access to the victim's memory space. The value written and the address written to may be controlled by the attacker to various degrees. The primitive, for example, may allow:

- writing a fixed value to an attacker-controlled address, or
- writing to an address consisting of a fixed base and an attacker-controlled offset limited to a specific range (e.g. a 32-bit offset) , or
- writing to an attacker-controlled base address with a fixed offset.

Consider describing in more detail why the range limitation matters#162

Primitives can be further classified according to more detailed properties. See slide 11 of (Miller, n.d.) for an example.

The most powerful version of a write primitive is an *arbitrary write* primitive, where both the address and the value are fully controlled by the attacker.

A *read primitive*, respectively, gives the attacker read access to the victim's memory space. The address of the memory location accessed will be controlled by the attacker to some degree, as for the write primitive. A particularly useful primitive is an *arbitrary read* primitive, in which the address is fully controlled by the attacker.

The effects of a write primitive are perhaps easier to understand, as it has obvious side-effects: a value is written to the victim program's memory. But how can an attacker observe the result of a read primitive?

This depends on whether the attack is interactive or non-interactive (Hu et al. 2016).

- In an *interactive attack*, the attacker gives malicious input to the victim program. The malicious input causes the victim program to perform the read the attacker instructed it to, and to output the results of that read. This output could be any kind of output, for example a network packet that the victim transmits. The attacker can observe the result of the read primitive by looking at this output, for example parsing this network packet. This process then repeats: the attacker sends more malicious input to the victim, observes the output and prepares the next input. You can see an example of this type of attack in (Beer 2020), which describes a zero-click radio proximity exploit.
- In a *non-interactive (one-shot) attack*, the attacker provides all malicious input to the victim program at once. The malicious input triggers multiple primitives one after the other, and the primitives are able to observe the effects of the preceding operations through the victim program's state. The input could be, for example, in the form of a JavaScript program (Groß 2020), or a PDF file pretending to be a GIF (Beer and Groß 2021).

The references in this section describe complicated modern exploits. Consider linking to simpler exploits, as well as some tutorial-level material. #163

How does an attacker obtain these kinds of primitives in the first place? The details vary, and in some cases it takes a combination of many techniques, some of which are out of scope for this book. But we will be describing a few of them in this chapter. For example a stack buffer overflow results in a (restricted) write primitive when the input size exceeds what the program expected.

As part of an attack, the attacker will want to execute each primitive more than once, since a single read or write operation will rarely be enough to achieve their end goal (more on this later). How can primitives be combined to perform multiple reads/writes?

In the case of an interactive attack, preparing and sending input to the victim

program and parsing the output of the victim program are usually done in an external program that drives the exploit. The attacker is free to use a programming language of their choice, as long as they can interact with the victim program in it. Let's assume, for example, an exploit program in C, communicating with the victim program over TCP. In this case, the primitives are abstracted into C functions, which prepare and send packets to the victim, and parse the victim's responses. Using the primitives is then as simple as calling these functions. These calls can be easily combined with arbitrary computations, all written in C, to form the exploit.

For this cycle of repeated input/output interactions to work, the state of the victim program must not be lost between the different iterations of providing input and observing output. In other words, the victim process must not be restarted.

It's interesting to note that while the read/write primitives consist of carefully constructed inputs to the victim program, the attacker can view these inputs as *instructions* to the victim program. The victim program effectively implements an interpreter unintentionally, and the attacker can send instructions to this interpreter. This is explored further in (Dullien 2020).

In the case of a non-interactive attack, all computation happens within the victim program. The duality of input data and code is even more obvious in this case, as the malicious input to the victim can be viewed as the exploit code. There are cases for which the input is obviously interpreted as code by the victim application as well, as in the case of a JavaScript program given as input to a JavaScript engine. In this case, the read/write primitives would be written as JavaScript functions, which when called have the unintended side-effect of accessing arbitrary memory that a JavaScript program is not supposed to have access to. The primitives can be chained together with arbitrary computations, also expressed in JavaScript.

There are, however, cases where the correspondence between data and code isn't as obvious. For example, in (Beer and Groß 2021), the malicious input consists of a PDF file, masquerading as a GIF. Due to an integer overflow bug in the PDF decoder, the malicious input leads to an unbounded buffer access, therefore to an arbitrary read/write primitive. In the case of JavaScript engine exploitation, the attacker would normally be able to use JavaScript operations and perform arbitrary computations, making exploitation more straightforward. In this case, there are no scripting capabilities officially supported. The attackers, however, take advantage of the compression format intricacies to implement a small computer architecture, in thousands of simple commands to the decoder. In this way, they effectively *introduce* scripting capabilities and are able to express their exploit as a program to this architecture.

So far, we have described read/write primitives. We have also discussed how an attacker might perform arbitrary computations:

- in an external program in the case of interactive attacks, or
- by using scripting capabilities (whether originally supported or introduced by the attacker) in non-interactive attacks.

Assuming an attacker has gained these capabilities, how can they use them to

achieve their goals?

The ultimate goal of an attacker may vary: it may be, among other things, getting access to a system, leaking sensitive information or bringing down a service. Frequently, a first step towards these wider goals is arbitrary code execution within the victim process. We have already mentioned that the attacker will typically have arbitrary computation capabilities at this point, but arbitrary code execution also involves things like calling arbitrary library functions and performing system calls.

Some examples of how the attacker may use the obtained primitives:

- Leak information, such as pointers to specific data structures or code, or the stack pointer.
- Overwrite the stack contents, e.g. to perform a ROP attack.
- Overwrite non-control data, e.g. authorization state. Sometimes this step is sufficient to achieve the attacker's goal, bypassing the need for arbitrary code execution.

Once arbitrary code execution is achieved, the attacker may need to exploit additional vulnerabilities in order to escape a process sandbox, escalate privilege, etc. Such vulnerability chaining is common, but for the purposes of this chapter we will focus on:

- Preventing memory vulnerabilities in the first place, thus stopping the attacker from obtaining powerful read/write primitives.
- Mitigating the effects of read/write primitives, e.g. with mechanisms to maintain Control-Flow Integrity (CFI).

2.3 Stack buffer overflows

A buffer overflow occurs when a read from or write to a data buffer exceeds its boundaries. This typically results in adjacent data structures being accessed, which has the potential of leaking or compromising the integrity of this adjacent data.

When the buffer is allocated on the stack, we refer to a stack buffer overflow. In this section we focus on stack buffer overflows since, in the absence of any mitigations, they are some of the simplest buffer overflows to exploit.

The stack frame of a function includes important control information, such as the saved return address and the saved frame pointer. Overwriting these values unintentionally will typically result in a crash, but the overflowing values can be carefully chosen by an attacker to gain control of the program's execution.

Example 1 A simple stack buffer overflow.

Here is a simple example of a program vulnerable to a stack buffer overflow[1]*:*

```
#include <stdio.h>
#include <string.h>
```

[1]This is an oversimplified example for illustrative purposes. However, as this is a wide class of vulnerabilities, many real-world examples can be found and studied.

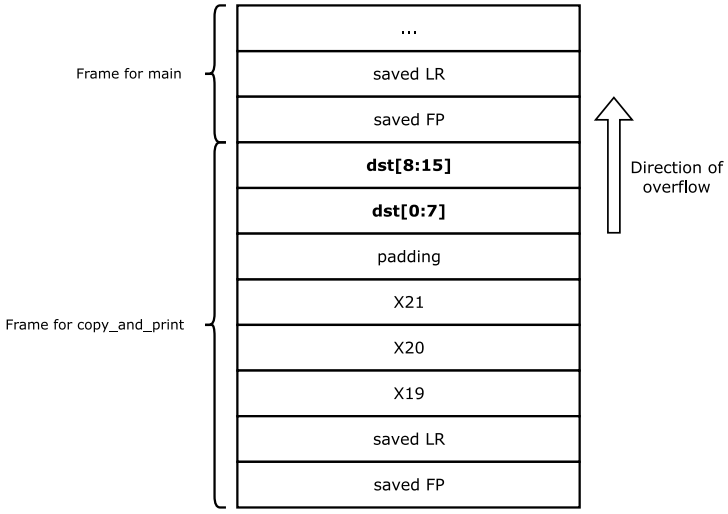
```c
void copy_and_print(char* src) {
  char dst[16];

  for (int i = 0; i < strlen(src) + 1; ++i)
    dst[i] = src[i];
  printf("%s\n", dst);
}

int main(int argc, char* argv[]) {
  if (argc > 1) {
    copy_and_print(argv[1]);
  }
}
```

In the code above, since the length of the argument is not checked before copying it into `dst`*, we have a potential for a buffer overflow.*

When looking at code generated for AArch64 with GCC 11.2[2]*, the stack layout looks like this:*



Stack frame layout for stack buffer overflow example

The exact details of the stack frame layout, including the ordering of variables and the exact control information stored, will depend on the specific compiler version you use and the architecture you compile for.

As can be seen the stack diagram, an overflowing write in function `copy_and_print` *can overwrite the saved frame pointer (FP) and link register (LR) in* `main`*'s frame. When* `copy_and_print` *returns, execution continues in* `main`*. When* `main` *returns, however, execution continues from the address stored in the saved LR, which has been overwritten. Therefore, when an attacker can choose the value that overwrites the saved LR, it's possible to*

[2]The code is generated with the `-fno-stack-protector` option, to ensure GCC's stack guard feature is disabled. We also used the `-O1` optimization level.

control where the program resumes execution after returning from `main`.

Before non-executable stacks were mainstream, a common way to exploit these vulnerabilities would be to use the overflow to simultaneously write shellcode[3] to the stack and overwrite the return address so that it points to the shellcode. (Aleph One 1996) is a classic example of this technique.

The obvious solution to this issue is to use memory protection features of the processor in order to mark the stack (along with other data sections) as non-executable[4]. However, even when the stack is not executable, more advanced techniques can be used to exploit an overflow that overwrites the return address. These take advantage of code that already exists in the executable or in library code, and will be described in the next section.

Stack canaries are an alternative mitigation for stack buffer overflows. The general idea is to store a known value, called the stack canary, between the buffer and the control information (in the example, the saved FP and LR), and to check this value before leaving the function. Since an overflow that would overwrite the return address is going to overwrite the canary first, a corruption of the return address through a stack buffer overflow will be detected.

This technique has a few limitations: first of all, it specifically aims to protect against stack buffer overflows, and does nothing to protect against stronger primitives (e.g. arbitrary write primitives). Control-flow integrity techniques, which are described in the next section, aim to protect the integrity of stored code pointers against any modification.

Secondly, since a compiler needs to generate additional instructions for ensuring the canary's integrity, heuristics are usually employed to determine which functions are considered vulnerable. The additional instructions are then generated only for the functions that are considered vulnerable. Since heuristics aren't always perfect, this poses another potential limitation of the technique. To address this, compilers can introduce various levels of heuristics, ranging from applying the mitigations only to a small proportion of functions, to applying it universally. See, for example, the `-fstack-protector`, `-fstack-protector-strong` and `-fstack-protector-all` options offered by both GCC and Clang.

Another limitation is the possibility of leaks of the canary value. The canary value is often randomized at program start but remains the same during the program's execution. An attacker who manages to obtain the canary value at some point might, therefore, be able to reuse the leaked canary value and corrupt control information while avoiding detection. Choosing a canary value that includes a null byte (the C-style string terminator) might help in limiting the damage of overflows coming from string manipulation functions, even when the value is leaked.

Many buffer overflow vulnerabilities result from the use of unsafe library functions, such as `gets`, or from the unsafe use of library functions such as `strcpy`. There is extensive literature on writing secure C/C++ code, for example (Seacord

[3] A shellcode is a short instruction sequence that performs an action such as starting a shell on the victim machine.

[4] Note that the use of nested functions in GCC requires trampolines which reside on an executable stack. The use of nested functions, therefore, poses a security risk.

2013) and (Dowd, McDonald, and Schuh 2006). A different approach to limiting the effects of overflows is library function hardening, which aims to detect buffer overflows and terminate the program gracefully. This involves the introduction of feature macros like `_FORTIFY_SOURCE` (Sharma 2014).

Finally, it's important to mention that not all buffer overflows aim to overwrite a saved return address. There are many cases where a buffer overflow can overwrite other data adjacent to the buffer, for example an adjacent variable that determines whether authorization was successful, or a function pointer that, when modified, can modify the program's control flow according to the attacker's wishes.

Some of these vulnerabilities can be mitigated with the measures described in this section, but often more general measures to ensure memory safety or Control-Flow Integrity are necessary. For example, in addition to the hardening of specific library functions, compilers can also implement automatic bounds checking for arrays where the array bound can be statically determined (`-fsanitize=bounds`), as well as various other "sanitizers". We will describe these measures in following sections.

2.4 Use After Free (UaF)

A *use after free* (UaF) occurs when a variable is used (read and/or written) after it has been freed. Although this description assumes manual memory management using malloc/new and free/delete (heap allocation), if we think about memory as a resource one may apply the same idea more broadly. For example, getting a reference to a variable in the stack and using it after it has ended its scope, or somehow getting access to freed memory from a garbage collector. It is important to note that, although seemingly related, some authors prefer to not mix the definitions. For example, the Common Weakness Enumeration (CWE) page only gives examples using raw malloc/frees, without any mentions to cases with stack or garbage collector. Unless explicitly stated, the rest of this section assumes raw memory management with malloc/new and free/delete.

Although some cases of UaF may just lead to unexpected software behavior or crashes, other cases may enable attackers to poison data and thereby alter program flow. There are many possibilities on how this can happen. Some of them depend on how the memory allocator manages its data. For example, if the attacker can trick the allocator to return the same address for two different allocation, that could lead to controllable data. This is shown in more detail in Example 2. For an overview of heap exploiting techniques, see (Kapil 2022).

Example 2 Simple UaF example.

```
struct auth_t {
  char name[32];
  int logged_in;
};

int main(int argc, char** argv) {
  char line[50];
```

```c
  while(1) {
    printf("[ auth = %p, service = %p ]\n", auth, service);

    if (fgets(line, sizeof(line), stdin) == NULL) break;

    // Usage: auth <name>
    if (strncmp(line, "auth ", 5) == 0) {
      auth = (struct auth_t*)malloc(sizeof(struct auth_t));

      // Memory is only set to 0 here, not on free[1]
      memset(auth, 0, sizeof(struct auth_t));

      if (strlen(line + 5) < 31) {
        strcpy(auth->name, line + 5);
      }
    }

    // Usage: reset
    else if (strncmp(line, "reset", 5) == 0) {
      // [1]
      free(auth);
    }

    // Usage: service <service-name>
    else if (strncmp(line, "service ", 8) == 0) {
      service = strdup(line + 8);
    }

    // Usage: login
    else if (strncmp(line, "login", 5) == 0) {
      // Possible use-after-free:
      if (auth && auth->logged_in) {
        printf("You are already logged in!\n");
      } else {
        printf("NOT AUTHORIZED!\n");
      }
    }
  }

  return 0;
}
```

The exploit below may not work in every system, as it assumes that calling malloc+free+malloc will result in both calls to malloc returning the same pointer. The example execution below tricks the software into thinking the user is logged in, by taking advantage of the UaF to change the boolean:

```
> auth admin
[ auth = 0x78322e1000, service = 0x0 ]
> reset
```

```
[ auth = 0x78322e1000, service = 0x0 ]
> service aaaaaaaaa0aaaaaaaaa0aaaaaaaaa0121
[ auth = 0x78322e1000, service = 0x78322e1000 ]
> login
You are already logged in!
```

In this example, four commands are issued:

1) *`auth admin` will allocate the `auth_t` structure for the first time. At this point, the user has a name, but is not authorized (`logged_in` is `false`).*
2) *`reset` will free the memory (but pointer to `auth` is not set to `nullptr`). `service (...)` will allocate a new string, potentially at the same address where the `auth_t` structure was previously allocated.*
3) *`service aaaaaaaaa0aaaaaaaaa0aaaaaaaaa0121` will end up setting the memory previously pointing to an `auth_t`.*
4) *`login` will use the dangling `auth_t` pointer. If this memory has been reallocated to the attacker controlled string, it will appear as if the field `name` is set to `aaaaaaaaa0aaaaaaaaa0aaaaaaaaa012` and the boolean `logged_in` to `true`.*

Detecting UaFs is usually not an easy task, as it depends not only on user inputs, but sometimes also on the execution flow. It can get even more complicated in multi-threaded environments. Therefore, many different UaF detection tools have been built, based on a number of different approaches:

Some detectors intercept calls to delete / free to inject a known value to the variables and then run the software looking for crashes that include that value. Another useful tool for detecting UaF is Arm's MTE (Memory Tagging Extension), discussed in section 2.8.

Fuzzing (generating random inputs) can also be useful, potentially used at the same time as other tools.

One could also include different algorithms to reduce the probability of exploiting use-after-free in allocators as a mitigation strategy, garbage collectors or reference-count based memory allocators.

Preventing UaF from happening may involve multiple approaches, depending on the context. From simple code changes, such as initializing allocated variables, to more elaborate changes, such as changing how the memory allocator works to avoid reusing specific memory locations. On top of that, decreasing relevance of UaF for attackers can be another interesting perspective (meaning, even if a UaF is present, decreasing the likelihood of it being exploitable). For example, MTE sync mode can force an application crash as soon as the UaF occurs, while Pointer Authentication (PAC) can be used to sign pointers so that even if they get poisoned, they cannot be used (more details in section 2.6.2.2.3).

Dive more into UaF detection and mitigations. Suggested starting points: Allocators using different algorithms to reduce the probability of exploiting use-after-free (this might be a whole section of it's own?)Type-aware allocation and deallocation functions, which explains the motivation for this feature at https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2719r5.html#a-concrete-use-case.Languages that use garbage collection or reference counting

Those are not exaustive lists of tools to detect, mitigate and prevent UaFs. But this section's goal was to give a brief introduction to the topic.

2.5 Code reuse attacks

In the early days of memory vulnerability exploitation, attackers could simply place shellcode of their choice in executable memory and jump to it. As non-executable stack and heap became mainstream, attackers started to reuse code already present in an application's binary and linked libraries instead. A variety of different techniques to this effect came to light.

The simplest of these techniques is return-to-libc (Solar Designer 1997). Instead of returning to shellcode that the attacker has injected, the return address is modified to return into a library function, such as `system` or `exec`. This technique is simpler to use when arguments are also passed on the stack and can therefore be controlled with the same stack buffer overflow that is used to modify the address.

2.5.1 Return-oriented programming

Return-to-libc attacks restrict an attacker to whole library functions. While this can lead to powerful attacks, it has also been demonstrated that it is possible to achieve arbitrary computation by combining a number of short instruction sequences ending in indirect control transfer instructions, known as **gadgets**. The indirect control transfer instructions make it easy for an attacker to execute gadgets one after another, by controlling the memory or register that provides each control transfer instruction's target.

In return-oriented programming (ROP) (Shacham 2007), each gadget performs a simple operation, for example setting a register, then pops a return address from the stack and returns to it. The attacker constructs a fake call stack (often called a ROP chain) which ensures a number of gadgets are executed one after another, in order to perform a more complex operation.

This will hopefully become more clear with an example: a ROP chain for AArch64 Linux that starts a shell, by calling `execve` with `"/bin/sh"` as an argument. The prototype of the `execve` library function, which wraps the exec system call, is:

```
int execve(const char *pathname, char *const argv[],
           char *const envp[]);
```

For AArch64, `pathname` will be passed in the `x0` register, `argv` will be passed in `x1`, and `envp` in `x2`. For starting a shell, it is sufficient to:

- Make `x0` contain a pointer to `"/bin/sh"`.
- Make `x1` contain a pointer to an array of pointers with two elements:
  - The first element is a pointer to `"/bin/sh"`.
  - The second element is zero (`NULL`).
- Make `x2` contain zero (`NULL`).

This can be achieved by chaining gadgets to set the registers `x0`, `x1`, `x2`, and then returning to `execve` in the C library.

Let's assume we have the following gadgets:

1. A gadget that loads `x0` and `x1` from the stack:

```
gadget_x0_x1:
  ldp x0, x1, [sp]
  ldp x20, x19, [sp, #64]
  ldp x29, x30, [sp, #32]
  ldr x21, [sp, #48]
  add sp, sp, #0x50
  ret
```

2. A gadget that sets `x2` to zero, but also clears `x0` as a side-effect:

```
gadget_x2:
  mov x2, xzr
  mov x0, x2
  ldp x20, x19, [sp, #32]
  ldp x29, x30, [sp]
  ldr x21, [sp, #16]
  add sp, sp, #0x30
  ret
```

Explain how these gadgets could result from C/C++ code. The current versions are slightly tweaked by hand to have more manageable offsets. #164

Both gadgets also clobber several uninteresting registers, but since `gadget_x2` also clears `x0`, it becomes clear that we should use a ROP chain that:

1. Returns to `gadget_x2`, which sets `x2` to zero.
2. Returns to `gadget_x0_x1`, which sets `x0` and `x1` to the desired values.
3. Returns to `execve`.

Figure 1 shows this control flow.

We can achieve this by constructing the fake call stack shown in figure 2, where "Original frame" marks the frame in which the address of `gadget_x2` has replaced a saved return address that will be loaded and returned to in the future. As an alternative, an attacker could place this fake call stack somewhere else, for example on the heap, and use a primitive that changes the stack pointer's value instead. This is known as stack pivoting.

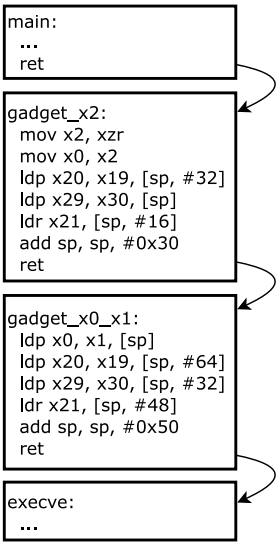


Figure 1: ROP example control flow

Note that this fake call stack contains zero bytes, even without considering the exact values of the various return addresses included. An overflow bug that is based on a C-style string operation would not allow an attacker to replace the stack contents with this fake call stack in one go, since C-style strings are null-terminated and copying the fake stack contents would stop once the first zero byte is encountered. The ROP chain would therefore need to be adjusted so that it doesn't contain zero bytes, for example by initially replacing the zero bytes with a different byte and adding some more gadgets to the ROP chain that write zero to those stack locations.

A question that comes up when looking at the stack diagram is "how do we know the addresses of these gadgets"? We will talk a bit more about this in the next section.

ROP gadgets like the ones used here may be easy to identify by visual inspection of a disassembled binary, but it's common for attackers to use "gadget scanner" tools in order to discover large numbers of gadgets automatically. Such tools can also be useful to a compiler engineer working on a code reuse attack mitigation, as they can point out code sequences that should be protected and have been missed.

Anything in executable memory can potentially be used as a ROP gadget, even if the compiler has not intended it to be code. This includes literal pools which are intermingled with code, and, on architectures with variable length instruction encoding, returning to the middle of an instruction. In a JIT compiler where the attacker might influence what literals are generated this can be particularly powerful. For example, on x86, the compiler might have emitted the instruction `mov $0xc35f, %ax` which is encoded as the four bytes `66 b8 5f c3`. If the attacker can divert execution two bytes into that 4-byte instruction it will execute `5f c3`. Those bytes corresponds to the two single byte instructions `pop`

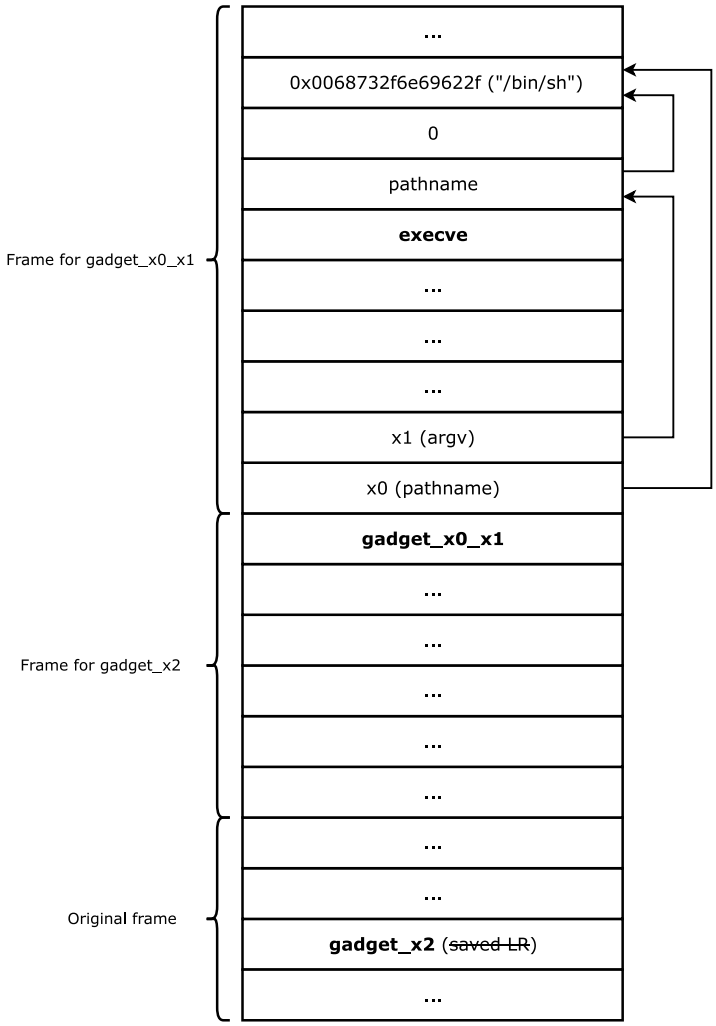


Figure 2: ROP example fake call stack

`%rdi; ret` which is a useful ROP gadget.

2.5.2 Jump-oriented programming

Jump-oriented programming (JOP) (Bletsch et al. 2011) is a variation on ROP, where gadgets can also end in indirect branch instructions instead of return instructions. The attacker chains a number of such gadgets through a dispatcher gadget, which loads pointers one after another from an array of pointers, and branches to each one in return. The gadgets used must be set up so that they branch or return back to the dispatcher after they're done. This is demonstrated in figure 3.

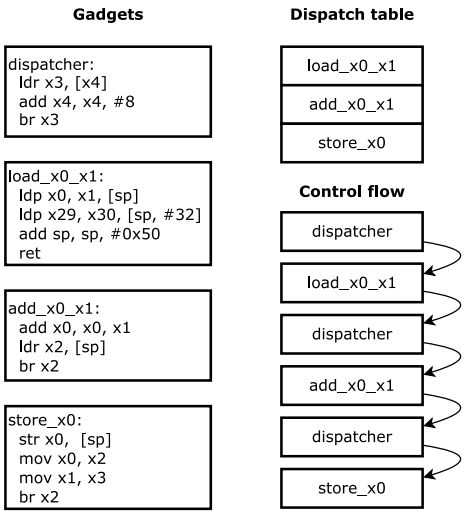


Figure 3: JOP example

The gadgets in the figure are made up, chosen to highlight that each gadget can end in a different type of indirect control flow transfer instruction. Consider replacing them with more realistic ones. #165

In figure 3, `x4` initially points to the "dispatch table", which has been modified by the attacker to contain the addresses of the three gadgets they want to execute. The dispatcher gadget loads each address in the dispatch table one by one and branches to them. The first gadget loads `x0` and `x1` from the stack, where the attacker has placed the inputs of their choice. It then loads its return address, also modified by the attacker so that it points back to the dispatcher gadget, and returns to it. The dispatcher branches to the next gadget, which adds `x0` and `x1` and leaves the result in `x0`, branching back to the dispatcher through another value loaded from the stack into `x2`. The final gadget stores the result of the addition, which remains in `x0`, to the stack, before branching to `x2`, which still points to the dispatcher gadget.

2.5.3 Counterfeit Object-oriented programming

Counterfeit Object-oriented programming (COOP) (Schuster et al. 2015) is a code reuse technique that takes advantage of C++ virtual function calls. A COOP attack takes advantage of existing virtual functions and vtables, and creates fake objects pointing to these existing vtables. The virtual functions used as gadgets in the attack are called vfgadgets. To chain vfgadgets together, the attacker uses a "main loop gadget", similar to JOP's dispatcher gadget, which is itself a virtual function that loops over a container of pointers to C++ objects and invokes a virtual function on these objects. (Schuster et al. 2015) describes the attack in more detail. It is specifically mentioned here as an example of an attack that doesn't depend on directly replacing return addresses and code pointers, like ROP and JOP do. Such language-specific attacks are important to consider when considering mitigations against code reuse attacks, which will be the topic of the next section.

It would be nice to have a small example of a COOP attack, similar to the JOP example in the previous section. #261

2.5.4 Sigreturn-oriented programming

One last example of a code reuse attack that is worth mentioning here is sigreturn-oriented programming (SROP) (Bosman and Bos 2014). It is a special case of ROP where the attacker creates a fake signal handler frame and calls `sigreturn`. `sigreturn` is a system call on many UNIX-type systems which is normally called upon return from a signal handler, and restores the state of the process based on the state that has been saved on the signal handler's stack by the kernel previously, on entry to the signal handler. The ability to fake a signal handler frame and call `sigreturn` gives an attacker a simple way to control the state of the program.

2.6 Mitigations against code reuse attacks

When discussing mitigations against code reuse attacks, it is important to keep in mind that there are two capabilities the attacker must have for such attacks to work:

- the ability to overwrite return addresses, function pointers or other code pointers.
- knowledge of the target addresses to overwrite them with (e.g. libc function entry points).

When code reuse attacks were first described, programs used to contain absolute code pointers, and needed to be loaded at fixed addresses. The stack base was predictable, and libraries were loaded in predictable memory locations. This made code reuse attacks simple, as all of the addresses needed for a successful exploit were easy to discover. In this section, we're going to discuss mitigations that make it harder for an attacker to obtain these capabilities.

The ability for an attacker to overwrite code pointers often boils down to the being able to overwrite them while they are stored in memory, rather than in machine registers. Overwriting value in machine registers directly is often not possible. Attackers use memory vulnerabilities to be able to overwrite pointers in memory. With that in mind, one could assume that code reuse mitigations are not necessary for programs written in memory-safe languages, as they should not have any memory vulnerabilities. However, most real-life programs written in memory-safe languages still contain at least portions of binary code written in unsafe languages. An attacker could obtain a write primitive in the unsafe portion of the program, and use it to overwrite code pointers in the memory-safe portion of the program. Therefore, mitigations against code reuse attacks are still relevant for programs written in memory-safe languages.

Another reason that attackers could obtain write primitives in memory-safe programs is due to bugs in the compiler or the runtime. This is especially true for JIT-based languages, see section 2.9 for more details.

2.6.1 ASLR

Address space layout randomization (ASLR) makes this more difficult by randomizing the positions of the memory areas containing the executable, the loaded libraries, the stack and the heap. ASLR requires code to be position-independent. Given enough entropy, the chance that an attacker would successfully guess one or more addresses in order to mount a successful attack will be greatly reduced.

Does this mean that code reuse attacks have been made redundant by ASLR? Unfortunately, this is not the case. There are various ways in which an attacker can discover the memory layout of the victim program. This is often referred to as an "info leak" (Serna 2012).

Since we can not exclude code reuse attacks solely by making addresses hard to guess, we need to also consider mitigations that prevent attackers from overwriting return addresses and other code pointers. Some of the mitigations described earlier, like stack canaries and library function hardening, can help in specific situations, but for the more general case where an attacker has obtained arbitrary read and write primitives, we need something more.

2.6.2 Control-flow Integrity (CFI)

Control-flow integrity (CFI) is a family of mitigations that aim to preserve the intended control flow of a program. This is done by restricting the possible targets of indirect branches and returns. A scheme that protects indirect jumps and calls is referred to as forward-edge CFI, whereas a scheme that protects returns is said to implement backward-edge CFI.

Ideally, a CFI scheme would not allow any control flow transfers that don't occur in a correct program execution. However, different schemes have varying granularities. In general, the legal branch targets will be divided into classes, with targets in each class treated as equivalent for security purpose. A branch is permitted to transfer control to any member of its intended target class.

CFI schemes are sometimes classified as coarse-grained or fine-grained. A coarse-

grained CFI scheme is one that uses a small number of large equivalence classes, whereas a fine-grained scheme uses a larger number of smaller classes, so that the possible branch targets from a given location are more restricted (perhaps at a greater performance cost).

For example, a CFI scheme that allows an indirect function call to continue the execution at the start of any function would be considered coarse-grained. If it instead restricted to the subset of functions with the appropriate type signature, it would be fine-grained.

Forward-edge CFI schemes often rely on function type checks or use static analysis (points-to analysis) to identify potential control flow transfer targets. (Burow et al. 2017) compares a number of available CFI schemes based on the precision. For forward-edge CFI schemes, for example, schemes are classified based on whether or not they perform, among others, flow-sensitive analysis, context-sensitive analysis and class-hierarchy analysis.

The next few subsections go into a bit more detail on the common CFI schemes. These CFI schemes are used in production to harden specific kinds of control flow transfers. They include:

- Clang CFI
- arm64e, see McCall and Bougacha (2019) and pauthabi, see Korobeynikov (2024)
- kcfi
- various shadow stacks
- pac-ret, see Cheeseman (2019)
- Arm BTI, Intel IBT
- Microsoft Control Flow Guard (CFG)

Table 2.1: A few of the key properties of the most common CFI schemes.

Name	forward-edge?	backward-edge?	fine-grained?	hardware-based?
Clang CFI	Yes	No	Yes	No
arm64e/pauthabi	Yes	Yes	Yes	Yes
kcfi	Yes	No	Yes	No
shadow stack	No	Yes	Yes	Depends
pac-ret	No	Yes	Yes	Yes
BTI, IBT	Yes	No	No	Yes
Control Flow Guard	Yes	No	No	No

There are many more CFI approaches, often academic, but many of them are not widely used in production. This book focuses mostly on the deployed CFI schemes.

2.6.2.1 General CFI principles

2.6.2.1.1 Protecting (forward) indirect function calls In practice, most in-production CFI schemes harden indirect function calls by partitioning all

functions present in the program into equivalence classes. Each function is assigned a single equivalence class.

For C code, most CFI schemes either put all functions into a single equivalence class, or partition functions based on their signature.

For example, arm64e and pauthabi put all C functions in a single equivalence class see McCall and Bougacha (2019). Examples of CFI schemes that partition C functions based on their signature include kcfi and clang cfi.

Example 3 Equivalence classes for C functions based on signature.

In C, consider the following three functions:

```
void f1(int a) { /* ... */ }
void f2(int* b) { /* ... */ }
void f3(int c) { /* ... */ }
```

Function `f1` and `f3` have the same signature, but `f2` has a different signature. A CFI scheme that partitions functions based on their signature will assign `f1` and `f3` to the same equivalence class, and `f2` to a different equivalence class.

Probably the main reason why some CFI schemes put all C functions in a single equivalence class, is that real-world C code quite often implicitly casts one C function pointer type to another. This is technically incorrect C code, but happens to work on most platforms not using fine-grained CFI. Example 4 illustrates this.

Example 4 C code mixing different function pointer types.

```
#include <stdlib.h>
int cmp_long(const long *a, const long *b) { return *a < *b; }
long sort_array(long *arr, long size) {
  /* The prototype of qsort is:
     void qsort(void *base, size_t nmemb, size_t size,
                int (*compar)(const void *, const void *)); */
  qsort(arr, size, sizeof(long), &cmp_long);
  return arr[0];
}
```

In this example, the function `cmp_long` has a different signature than the function pointer type expected by `qsort`.

This code will run under CFI schemes that put all C functions in a single equivalence class, but will fail under CFI schemes that partition C functions based on their signature.

2.6.2.1.2 Protecting (forward) virtual calls Many CFI schemes check that a C++ virtual function call happens on an object of the correct dynamic type. A few examples are: clang-cfi, arm64e, pauthabi.

Example 5 C++ virtual function call.

```
struct A {
  virtual void f();
};
```

```
struct B : public A {
  virtual void f();
};
struct C : public A {
  virtual void f();
};
void call_foo(A* a, B* b){
  a->f();
  b->f();
}
struct D : public B {
  virtual void f();
};
```

In this example, a very fine-grained CFI scheme should allow the call `a->f()` *if* `a` *is an instance of* `A`*,* `B`*,* `C` *or* `D`*. In other words, it should make sure either* `A::f`*,* `B::f`*,* `C::f` *or* `D::f` *gets called and no other function. Similarly, the call* `b->f()` *should only be allowed if it ends up calling either* `B::f` *or* `D::f`*, but not* `A::f` *or* `C::f`*.*

clang-cfi implements this very fine-grained CFI scheme when enabling the `-fsanitize=cfi-cast-strict` *option, whereas arm64e and pauthabi implement a more coarse-grained CFI scheme that only (probabilistically) checks whether any call to method* `f` *is one of the overloaded functions from* `A::f`*, i.e.* `A::f`*,* `B::f`*,* `C::f` *or* `D::f`*. This is less precise on the call* `b->f()` *above.*

2.6.2.1.3 Protecting (forward) switch jumps Switch statements with many cases whose values are densely packed together are often implemented using a jump table, which is an array of pointers or offsets to the code for each case. Ultimately, the address to jump to is computed by loading from the jump table, and then an indirect jump to the computed address is performed. If an attacker can control the value used to index into the jump table, they can make the jump target point to a different address, leading to the attacker taking over the control flow.

Most CFI schemes do not protect against this, but arm64e and pauthabi do, as explained in the example below. This is also explained in (McCall and Bougacha 2019, slide 39-40).

Example 6 jump table CFI hardening.

```
  switch (b) {
    case 0:
      return a+1;
    case 1:
      return a-5;
    case 2:

      ... /* cases 3-13 omitted for brevity */

    case 14:
      return a % 4;
```

```
    case 15:
      return a & 3;
    }
```

Arm64 generates the following assembly code for the jump table. The comments have been added manually for clarity.

```
  ;; x0 contains the value of b, which is the switch value.
  adrp  x8, lJTI0_0@PAGE
  add   x8, x8, lJTI0_0@PAGEOFF
  ;; x8 now contains the address of the jump table.
  adr   x9, LBB0_2
  ldrb  w10, [x8, x0]
  ;; w10 now contains the offset in words from LBB0_2
  ;; to the target instruction to jump to.
  add   x9, x9, x10, lsl #2
  ;; x9 now contains the address of the instruction to jump to.
  br    x9
  ;; code emitted for brevity

lJTI0_0:
  .byte (LBB0_2-LBB0_2)>>2
  .byte (LBB0_6-LBB0_2)>>2
  .byte (LBB0_11-LBB0_2)>>2
  .byte (LBB0_10-LBB0_2)>>2
  .byte (LBB0_9-LBB0_2)>>2
  ;; more cases omitted for brevity
```

In this sequence, if the value in `x0` *was loaded from memory, it could potentially be attacker controlled. If an attacker can control that value, they can make the jump target point to an almost arbitrary address, by loading a word offset value from any readable location in the process memory space.*

To prevent this, arm64e and pauthabi check that the value in `x0` *is in range before loading the jump table offset:*

```
  mov   x16, x0
  ;; check that x0 is in range
  cmp   x16, #15
  ;; if x0 is out of range, set switch value to zero (in x16)
  ;; this guarantees that the value will be loaded from the jump
  ;; table which is read-only and cannot be modified by an attacker
  csel  x16, x16, xzr, ls
  adrp  x17, lJTI0_0@PAGE
  add   x17, x17, lJTI0_0@PAGEOFF
  ldrsw x16, [x17, x16, lsl #2]
Ltmp1:
  adr   x17, Ltmp1
  add    x16, x17, x16
  br     x16
  ;; code emitted for brevity
```

```
lJTI0_0:
  .long LBB0_2-Ltmp1
  .long LBB0_6-Ltmp1
  .long LBB0_11-Ltmp1
  .long LBB0_10-Ltmp1
  .long LBB0_9-Ltmp1
```

2.6.2.1.4 Protecting (backward-edge) returns When a function is called, the address of the instruction after the call instruction is stored in a register or on the stack. That address of the next instruction is called the "return address". When the called function returns, it will use an instruction to branch to the return address. This is an indirect control flow, since the target of the branch isn't hard-coded in the instruction, but comes from a register or a memory location. If an attacker can change the value of the return address, they can redirect the control flow.

Example 7 Typical AArch64 call/return sequence.

```
  ...
  ;; the bl instruction jumps to function f and
  ;; stores the return address, i.e. the address of
  ;; the 'add' instruction, in register x30
  bl f
  add x0, x0, x1
  ...

f:
  ;; stores x29 and x30 on the stack. After executing this
  ;; instruction the return address is in memory, on the stack.
  stp x29, x30, [sp, #16]!
  ...
  ;; load the x29 and x30 registers from the stack. Under the usual
  ;; threat model, an attacker with a write primitive may have overwritten
  ;; the value in memory and may control the value in registers x29 and x30
  ldp x29, x30, [sp], #16
  ;; The return instruction jumps to the address stored in register x30.
  ret x30
```

Most backward-edge CFI schemes add checks before executing the return instruction to verify that the return address hasn't been tampered with.

[Shadow stack]]{.index entry="shadow stack"} approaches store the return address on a second stack. Some shadow stack approaches also store the return address in the original location in the normal stack. In those, before the return is executed, it verifies that the return value on both the regular stack and the shadow stack are equal. All shadow stack approaches have mechanisms to make it hard to impossible for an attacker to overwrite the return address on the shadow stack.

A software-only implementation is the clang shadow stack, which is explained in more detail in section 2.6.2.2.2. Hardware-supported shadow stacks include Arm's Guarded control stack (GCS), and Intel's CET Shadow Stack.

2.6.2.1.5 Other code pointers that may need protection Anytime a code pointer is stored in memory, it can potentially be modified by an attacker with a write primitive. The previous sections gave examples of how code pointers may originate from various source code constructs, such as function pointers, vtables, return addresses, etc. This list isn't exhaustive, and there are more source code constructs that can lead to code pointers being stored in memory, such as:

- C++ co-routines are typically implemented using structures containing code pointers. Abusing these has recently been coined as Coroutine Frame-Oriented Programming (CFOP).
- Procedure Linkage Table (PLT) and the Global Offset Table(GOT) often contain code pointers. One common way to protect these from being overwritten by an attacker is to make these tables read-only during program startup.
- Signal handlers and signal handler frames contain code pointers, see 2.5.4 for more details.

Should we list examples of indirect control flow from other languages too?

2.6.2.2 Detailed descriptions of a few CFI schemes

Below, we explore a few CFI schemes in more detail:

- Clang CFI in section 2.6.2.2.1
- Clang Shadow Stack in section 2.6.2.2.2
- Pointer Authentication-based CFI schemes:
  - pac-ret in section 2.6.2.2.3.1
  - arm64e and pauthabi in section 2.6.2.2.3.2
- Branch Target Identification (BTI) in section 2.6.2.2.4

2.6.2.2.1 Clang CFI Clang's CFI includes a variety of forward-edge control-flow integrity checks. These include checking that the target of an indirect function call is an address-taken function of the correct type.

When compiling with `-fsanitize=cfi -flto -fvisibility=hidden` [5], the code for `call_foo` would look something like this:

```
00000000004006b4 <call_foo(A*)>:
  4006b4:       a9bf7bfd        stp     x29, x30, [sp, #-16]!
  4006b8:       910003fd        mov     x29, sp
  4006bc:       f9400008        ldr     x8, [x0]
  4006c0:       90000009        adrp    x9, 400000 <_init-0x558>
  4006c4:       91216129        add     x9, x9, #0x858
  4006c8:       cb090109        sub     x9, x8, x9
  4006cc:       d1004129        sub     x9, x9, #0x10
  4006d0:       93c91529        ror     x9, x9, #5
  4006d4:       f100093f        cmp     x9, #0x2
  4006d8:       540000a2        b.cs    4006ec <call_foo(A*)+0x38>
  4006dc:       f9400108        ldr     x8, [x8]
  4006e0:       d63f0100        blr     x8
  4006e4:       a8c17bfd        ldp     x29, x30, [sp], #16
```

[5] The LTO and visibility flags are required by Clang's CFI.

```
4006e8:       d65f03c0        ret
4006ec:       d4200020        brk     #0x1
```

This code looks complicated, but what it does is check that the virtual table pointer (vptr) of the argument points to the vtable of `A` or of `B`, which are stored consecutively and are the only allowed possibilities. The checks generated for different types of control-flow transfers are similar.

2.6.2.2.2 Clang Shadow Stack Clang also implements a backward-edge CFI scheme known as Shadow Stack. In Clang's implementation, a separate stack is used for return addresses, which means that stack-based buffer overflows cannot be used to overwrite return addresses. The address of the shadow stack is randomized and kept in a dedicated register, with care taken so that it is never leaked, which means that an arbitrary write primitive cannot be used against the shadow stack unless its location is discovered through some other means.

As an example, when compiling with `-fsanitize=shadow-call-stack -ffixed-x18` [6], the code generated for the `main` function from the earlier stack buffer overflow example will look something like:

```
main:
    cmp w0, #2
    b.lt    .LBB1_2
    str x30, [x18], #8
    stp x29, x30, [sp, #-16]!
    mov x29, sp
    ldr x0, [x1, #8]
    bl  copy_and_print
    ldp x29, x30, [sp], #16
    ldr x30, [x18, #-8]!
.LBB1_2:
    mov w0, wzr
    ret
```

You can see that the shadow stack address is kept in `x18`. The return address is also saved on the "normal" stack for compatibility with unwinders, but it's not actually used for the function return.

2.6.2.2.3 Pointer Authentication In addition to software implementations, there are a number of hardware-based CFI implementations. A hardware-based implementation has the potential to offer improved protection and performance compared to an equivalent software-only CFI scheme.

One such example is Pointer Authentication (Rutland 2017), an Armv8.3 feature, supported only in AArch64 state, that can be used to mitigate code reuse attacks.

Pointer Authentication computes a pointer *signature* for a given address, called a Pointer Authentication Code (PAC), see figure 4. The PAC code is stored in the upper bits of the pointer which are otherwise unused.

[6] The `-ffixed-x18` flag results in treating the `x18` register as reserved, and is required by `-fsanitize=shadow-call-stack` on some platforms.

A pointer with a PAC code in the upper bits is called a *signed pointer*. A non-signed pointer is called a *raw pointer*.

The general idea behind Pointer Authentication is that attackers will try to overwrite a pointer in memory using a memory vulnerability. Pointer Authentication aims to detect when an attacker has overwritten a pointer in memory. It does this by making sure that pointers:

1. are always signed when they are in memory, and
2. between loading the pointer into a register and using it, the pointer is authenticated.

If the authentication fails, the program will fault.

Different hardening schemes are possible with pointer authentication, depending on which kinds of pointers get signed, such as only return addresses, all function pointers, or more.

An essential aspect of pointer authentication being useful is to make it hard for an attacker to construct the correct PAC that will pass authentication. To achieve that, next to the address, 2 other inputs are used to compute the PAC: a so-called key and a modifier:

- The key is a secret value that is not directly accessible to software, so that an attacker cannot retrieve the key value. This makes it hard for an attacker to compute the PAC value for a given address off-line. The key can be thought of as a pepper

- We also want to avoid that an attacker could take a signed pointer from one context in your program and use it in a different context. The modifier is a value that is specific to the context in which the pointer is used. Different hardening schemes will use different modifiers. Two examples of different hardening schemes built on top of Pointer Authentication are described in sections 2.6.2.2.3.1 (the pac-ret hardening scheme) and 2.6.2.2.3.2 (the arm64e/pauthabi hardening scheme). The modifier can be thought of as a salt.

  When an attacker successfully takes a signed pointer from one context and overwrites another pointer in another context with it, this is called a pointer substitution attack. Using different modifiers for different contexts makes pointer substitution attacks harder.

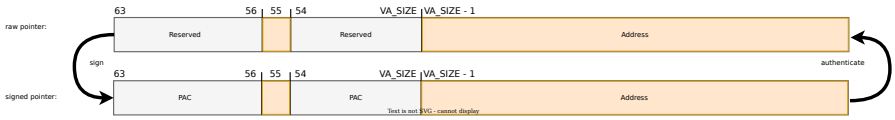


Figure 4: AArch64 sign and authenticate operations to convert raw pointers to signed pointers and vice versa

Pointer authentication instructions as described above can be used to implement a wide variety of hardening schemes. In this book, we only cover the two that are used in production on billions of devices in more detail: pac-ret and arm64e/pauthabi.

Other hardening schemes based on Pointer Authentication which we're not covering further include: PACStack (Liljestrand et al. 2021), Camouflage (Denis-Courmont et al. 2021), PAL (Yoo et al. 2021), PTAuth (farkhani, Ahmadi, and Lu 2021), PAC it up (Liljestrand et al. 2019), FIPAC (Schilling, Nasahl, and Mangard 2022), structure protection and more. Some of these harden binaries against attacks also in other ways than protecting control flow.

2.6.2.2.3.1 pac-ret: Backward-Edge CFI Clang and GCC both use Pointer Authentication for return address signing, when compiling with the `-mbranch-protection=pac-ret` flag. How it works is easiest to explain by example:

Example 8 pac-ret example.

When compiling the `main` function from example 1 with `pac-ret` enabled, the compiler will produce:

```
main:
    cmp     w0, #2
    b.lt    .LBB1_2
    paciasp
    stp     x29, x30, [sp, #-16]!
    ldr     x0, [x1, #8]
    mov     x29, sp
    bl      copy_and_print
    ldp     x29, x30, [sp], #16
    autiasp
.LBB1_2:
    mov     w0, wzr
    ret
```

Notice the `paciasp` and `autiasp` instructions. At entry to this function, the return address, i.e. the address the function will jump back to when executing the `ret` instruction at the end, is stored in register `x30`.

The instruction `paciasp` computes a PAC for the return address in register `x30`, and stores it in the upper bits of `x30`. The PAC is computed from the following "inputs":

1. *The address in `x30`, which is the return address,*
2. *Secret key `IA`, as indicated by the `ia` in instruction `paciasp`. That key is not accessible by the program.*
3. *As a modifier, the current value of the stack pointer (`sp`), as indicated by `sp` in the instruction `paciasp`.*

After the `paciasp` instruction, the value in `x30` is a signed pointer. The `stp` instruction stores the signed pointer to memory. Under the usual threat model, an attacker with a write primitive can modify the value while it is in memory. Therefore, after the value is loaded into `x30` again, by the `ldp` instruction, it should be considerded to be potentially tampered with.

Therefore, the compiler inserts the `autiasp` instruction between loading the signed pointer from memory and using it in the `ret` instruction. The `autiasp`

instruction verifies the PAC in the upper bits of `x30`, taking into account the secret key `IA` and modifier `sp`. If the PAC is correct, which will be the case in normal execution, the extension bits of the address are restored, so that the address can be used in the `ret` instruction. However, if the PAC is incorrect, the upper bits will be corrupted so that subsequent uses of the address (such as in the `ret` instruction) will result in a fault.

By making sure we don't store any return addresses without a PAC, we can significantly reduce the effectiveness of ROP attacks: since the secret key is not retrievable by an attacker, an attacker cannot calculate the correct PAC for a given address and modifier, and is restricted to guessing it.

The probability of success when guessing a PAC depends on the exact number of PAC bits available in a given system configuration.

The authenticated pointers are vulnerable to pointer substitution attacks, where a pointer that has been signed with a given modifier is replaced with a different pointer that has also been signed with the same modifier. In the `pac-ret` scheme, this is mitigated by using the stack pointer as the modifier, which limits reuse of signed return address pointers to function frames that happen to have the same stack pointer value.

2.6.2.2.3.2 arm64e and pauthabi: Forward-Edge CFI

The use of pauth in arm64e or pauthabi should be explained in more detail, including the concepts of signing and authentication oracles #259

Pointer Authentication can also be used more widely, for example to implement a forward-edge CFI scheme, as is done in the arm64e ABI (McCall and Bougacha 2019). The Pointer Authentication instructions, however, are generic enough to also be useful in implementing more general memory safety measures, beyond CFI.

2.6.2.2.4 BTI and other coarse-grained CFI schemes Branch Target Identification (BTI), introduced in Armv8.5, offers coarse-grained forward-edge protection. With BTI, the locations that are targets of indirect branches have to be marked with a new instruction, `BTI`. There are four different types of BTI instructions that permit different types of indirect branches (indirect jump, indirect call, both, or none). An indirect branch to a non-BTI instruction or the wrong type of BTI instruction will raise a Branch Target Exception.

Both Clang and GCC support generating BTI instructions, with the `-mbranch-protection=bti` flag, or, to enable both BTI and return address signing with Pointer Authentication, `-mbranch-protection=standard`.

Two aspects of BTI can simplify its deployment: individual pages can be marked as guarded or unguarded, with BTI checks as described above only applying to indirect branches targeting guarded pages. In addition to this, the BTI instruction has been assigned to the hint space, therefore it will be executed as a no-op in cores that do not support BTI, aiding its adoption.

Another implementation of coarse-grained forward-edge CFI is Windows Control Flow Guard, which only allows indirect calls to functions that are marked as valid indirect control flow targets.

2.6.2.3 CFI implementation pitfalls

When implementing CFI measures like the ones described here, it is important to be aware of known weaknesses that affect similar schemes. (Conti et al. 2015) describes how CFI implementations can suffer when certain registers are spilled on the stack, where they could be controlled by an attacker. For example, if a register that contains a function pointer that has just been validated gets spilled, the check can effectively be bypassed by overwriting the spilled pointer.

Having discussed various mitigations against code reuse attacks, it's time to turn our attention to a different type of attacks, which do not try to overwrite code pointers: attacks against non-control data, which will be the topic of the next section.

2.7 Non-control data attacks

In the previous sections, we have focused on subverting control flow by overwriting control data, which are used to change the value of the program counter, such as return addresses and function pointers. Since these types of attacks are prominent, many mitigations have been designed with the goal of maintaining control-flow integrity. Non-control data attacks, also known as data-only attacks, can completely bypass these mitigations, since the data they modify is not the control data that these mitigations protect.

Non-control data attacks can range from very simple attacks targeting a single piece of data to very elaborate attacks with very high expressiveness (Beer and Groß 2021). A very simple example may look something like this:

```
// Returns zero for failure, non-zero for success.
int authenticate() {
  int authenticated = 0;
  char passphrase[10];
  if (fgets(passphrase, 20, stdin)) {     // buffer overflow
    if (!strcmp(passphrase, "secret\n")) {
      authenticated = 1;
    }
  }
  return authenticated;
}
```

The example shows a simplified[7] function that reads a passphrase from a user, compares it with a known value and sets an integer stack variable to indicate whether "authentication" was successful or not. The function contains a very

[7] This is obviously not a realistic example of how authentication should be done, but simply serves to illustrate how a buffer overflow into a non-control variable can have serious security consequences.

obvious buffer overflow, as the string length limit passed to `fgets` does not match the buffer size.

Figure 5 shows the stack frame layout for this function when the code is compiled for AArch64 with Clang 10.0[8]. As the figure shows, an overflow of `passphrase` will overwrite `authenticated`, setting it to a non-zero value, even though the passphrase was incorrect. The `authenticate` function will then return a non-zero value, incorrectly indicating authentication success.

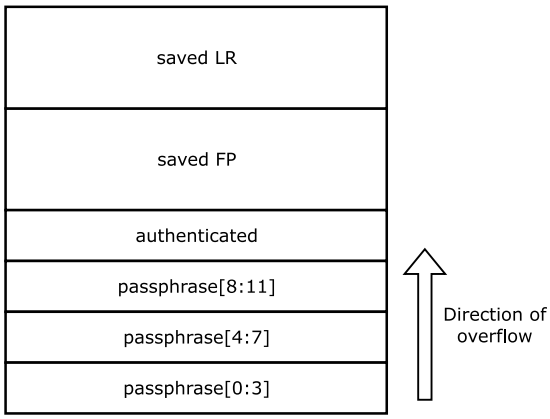


Figure 5: Stack frame for `authenticate`

For many more simple examples of data-only attacks that can occur in real applications, see (Chen et al. 2005). Although this makes it clear that data-only attacks are a real issue, it leaves open a very important question: what are the limits of such attacks? It is tempting to assume that data-only attacks are somehow inherently limited, however it has been demonstrated in (Hu et al. 2016) that they can, in fact, be very expressive. (Hu et al. 2016) describes Data-Oriented Programming (DOP), a general method for building data-only attacks against a vulnerable program, starting from a known memory error in the program[9].

The authors of (Hu et al. 2016) describe a small language called MINDOP, with a virtual instruction set and virtual registers. The virtual registers of MINDOP correspond to memory locations. The MINDOP instructions correspond to operations on these virtual registers, for example loading a value into a virtual register, storing a value from a virtual register, arithmetic operations and even conditional and unconditional jumps. The authors show how to identify gadgets in the code that implement the various MINDOP instructions and are reachable from memory errors, and how those gadgets can be stitched together with the help of dispatcher gadgets, the role of which is specifically to chain gadgets

[8]The stack frame layout may be significantly different for other architectures and compilers.

[9]The authors describe how DOP gadgets can be chained to simulate a Turing machine, making DOP attacks Turing-complete (it's not possible to simulate the infinite tape of a Turing machine on any actual hardware, of course). Turing-completeness is not, however, a particularly useful measure of exploitability, as explained in (Flake 2018). Many applications offer their users the ability to perform arbitrary computation, for example JavaScript engines, and those capabilities can be useful to an attacker, but performing a computation without affecting normal program behavior does not constitute "exploitation".

together.

Stitching gadgets together is simpler for interactive attacks, where the attacker can keep providing malicious input to trigger the initial memory error and a certain chain of gadgets, as many times as needed. For non-interactive attacks, the MINDOP jump operations are required as well, used in conjunction with a memory location that provides a virtual program counter.

The process of creating a DOP attack is not so simple and not fully automated. Related literature (Ispoglou et al. 2018) focuses on automating data-only attacks.

When reading write-ups on recent security issues, instead of terminology related to data-oriented gadgets, you are more likely to encounter the term "primitive", which has been described in an earlier section. These concepts are related: an arbitrary read primitive, for example, can be produced by chaining a (possibly large) number of DOP gadgets. Talking about primitives offers a nicer level of abstraction, as it tends to be simpler to reason in terms of higher-level operations instead of many small pieces of code that need to be stitched together to perform the operations.

To summarize, data-only attacks are a significant concern. As most of the mitigation techniques we have seen so far are control-flow oriented, they are by design inadequate to protect against this different type of attacks. In the next section, we will look at what we can do to address them at their source: memory errors.

2.8 Preventing and detecting memory errors

We have so far discussed how languages that are not memory safe, like C and C++, are vulnerable to memory errors and therefore exploitation. In this section, we will discuss tools that are available to C/C++ programmers to help them detect vulnerabilities that can lead to memory errors.

2.8.1 Sanitizers

Sanitizers are tools that detect bugs during program execution. Sanitizers usually have two components: a compiler instrumentation part that introduces the new checks, and a runtime library part. They are often too expensive to run in production mode, as they tend to increase execution time and memory usage. They are commonly used during testing of an application, frequently in combination with fuzzers[10].

A very popular sanitizer is Address Sanitizer (ASan). It aims to detect various memory errors. These include out-of-bounds accesses, use-after-free, double-free and invalid free[11]. There are Address Sanitizer implementations for both GCC and Clang, but we will focus on the Clang implementation here.

ASan uses shadow memory to keep track of the state of the application's memory. Each byte of shadow memory records information on 8 bytes of the application's

[10] Fuzzing is a powerful testing technique that relies on automatically generating large amounts of random inputs to the program under test.

[11] ASan also includes a memory leak detector.

memory. It represents how many of the 8 bytes are addressable. When none of the bytes are addressable, it encodes additional details (whether the 8 bytes are out-of-bounds stack, out-of-bounds heap, freed memory, and so on). Requiring one byte of shadow memory for every 8 bytes of application memory means that ASan needs to reserve one-eighth of the application's virtual address space (Serebryany et al. 2012). Shadow memory is allocated in one contiguous chunk, which keeps mapping application memory to shadow memory simple.

ASan's runtime library replaces memory allocation functions like `malloc` and `free` with its own specialized versions. `malloc` introduces redzones before and after each allocation, which are marked as unaddressable. `free` marks the entire allocation as unaddressable and places it in quarantine, so that it doesn't get reallocated for a while (in a FIFO basis). This allows for detecting use-after-free. The runtime library also handles management of the shadow memory.

ASan's code instrumentation in the compiler introduces redzones around each stack array allocation, and around globals. It then instruments loads and stores to check whether the accessed memory is addressable, based on the information stored in the shadow memory, and reports an error if unaddressable memory is accessed.

ASan doesn't produce false positives and is easy to use. It requires compiling and linking a program with the `-fsanitize=address` option. It is used in practice for testing large projects. There is a similar tool for dynamic memory error detection in the Linux kernel, KASAN.

ASan's biggest drawback is its high runtime overhead and memory usage, due to the quarantine, redzones and shadow memory. Hardware-assisted AddressSanitizer (HWASAN) works similarly to ASan, but with partial hardware assistance can result in lower memory overheads, at the cost of being less portable.

On AArch64, HWASAN uses Top-Byte Ignore (TBI). When TBI is enabled, the top byte of a pointer is ignored when performing a memory access, allowing software to use that top byte to store metadata, without affecting execution. Each allocation is aligned to 16 bytes and each 16-byte chunk of memory (called "granule") is randomly assigned an 8-bit tag. The tag is stored in shadow memory and is also placed in the top byte of the pointer to the object. Memory loads and stores are then instrumented to check that the tag stored in the pointer matches the tag stored in memory, and report an error when a mismatch happens.

Add diagram to demonstrate how HWASAN works #168

For granules shorter than 16 bytes, the value stored in shadow memory is not the actual tag, but the length of the granule. The actual tag is stored at the last byte of the granule itself. For tags in shadow memory with values between 1 and 15, HWASAN checks that the access is within the bounds of the granule and the pointer tag matches the tag stored at the last byte of the granule.

HWASAN is also easy to use, and simply requires compiling and linking an application with the `-fsanitize=hwaddress` flag.

MemTagSanitizer goes one step further and uses the Armv8.5-A Memory Tagging Extension (MTE). With MTE, the tag checking is done automatically by hardware, and an exception is raised on mismatch. MTE's granule size is 16 bits, whereas tags are 4-bit.

Consider adding a whole section on MTE and its applications #169

UndefinedBehaviorSanitizer (UBSan) detects undefined behavior during program execution, for example array out-of-bounds accesses for statically determined array bounds, null pointer dereference, signed integer overflow and various kinds of integer conversions that result in data loss. Although some of these checks are not directly related to memory errors, these kinds of errors can lead to incorrect pointer arithmetic, incorrect allocation sizes, and other issues that lead to memory errors, so it is important to detect them and address them.

UBSan's documentation describes the full list of available checks. The majority of these checks are enabled with the `-fsanitize=undefined` flag, but there are also other useful groupings of checks, for example `-fsanitize=integer` for checks related to integer conversions and arithmetic.

There are many other sanitizers, more than can reasonably be covered in this section. For the interested reader, we list a few more:

- MemorySanitizer: detects uninitialized reads.
- ThreadSanitizer: detects data races.
- GWP-ASan: detects use-after-free and heap buffer overflows, with low overhead that makes it suitable for production environments. It performs checks only on a sample of allocations.

Describe other mechanisms for detecting memory errors, both software-based (static analysis, library and buffer hardening) and hardware-based, e.g. PAuth-based pointer integrity schemes, MTE etc #170

2.8.2 Bounds checking

Making sure that memory accesses happen within the bounds of each object's allocation is a very important part of memory safety. This is usually described with the term "spatial memory safety". Out-of-bounds accesses result in restricted read/write primitives[12]. An attacker can often easily convert these into arbitrary read/write primitives. For example, this can be achieved by overwriting pointer fields in allocations following the object that was the target of the problematic memory access.

The C and C++ memory languages do not, as a general rule, perform bounds checking[13]. This is one of the sources of memory errors in C/C++ programs. However, compilers have a history of introducing bounds checks, even though the language does not require them, in an effort to improve security of existing C/C++ codebases.

One of the simplest compiler options is `-Warray-bounds`, which warns when an array access is always out of bounds. This is therefore restricted to arrays with statically known size. This option is supported by both GCC and Clang.

Another option supported by both compilers is `-fsanitize=bounds`, included

[12] These primitives are restricted since they can only access a limited number of bytes past the end of the allocation.

[13] Some C++ containers have accessors that do perform bounds checking, for example `std::array::at()` and `std::vector::at()`.

in UBSan, which checks the bounds for accesses to statically sized arrays at runtime. This handles more cases than `-Warray-bounds`, as it can also check accesses to dynamic indices. However, it's still limited, as it cannot perform bounds checks on dynamically sized arrays, and it is still restricted to array bounds checking. A more comprehensive solution would also cover pointers in general, especially if pointer arithmetic is performed.

You may notice that there is a bit of overlap between the bounds checks introduced by `-fsanitize=bounds` and the Address Sanitizer. Although the scope of `-fsanitize=bounds` is restricted to statically sized arrays, it's interesting to note that it can still catch intra-object overflows on array member accesses that the Address Sanitizer would not, because the access is still technically within the allocation. For example, given the following code:

```
struct foo {
  int a[6];
  int b;
};

int get(struct foo *x, int i) {
  return x->a[i];
}
```

a call to `get(f, 6)` will give an error with `-fsanitize=bounds`, but not with `-fsanitize=address`.

Clang and GCC also support two builtin functions that return information on the size of a variable. `__builtin_object_size` can be used for objects of statically known size and always evaluates at compile time, whereas `__builtin_dynamic_object_size` can also propagate dynamic information from allocation functions that have been marked with the `alloc_size` function attribute. These two builtins can be then used to introduce bounds checks in user or library code. For example, the `_FORTIFY_SOURCE` macro instructs `glibc` to introduce bounds checks in various string and memory manipulation functions, such as `memcpy`. The number of checks increases as the value of the macro increases (the used values are currently 1-3). For example, the lower two levels won't use the `__builtin_dynamic_object_size` builtin, as it has a runtime overhead, additional to that of the checks themselves.

In order to support bounds checking for dynamically sized arrays, a recent proposal for GCC and Clang proposes the addition of a struct member attribute, `element_count`. This attribute will apply to flexible array members in structs, indicating another member of the struct that expresses the array's length.

The `-fbounds-safety` proposal goes a bit further, introducing a similar annotation that can be applied to pointers more generally. The proposal also aims to reduce the annotation burden placed on programmers by only requiring the annotations at Application Binary Interface (ABI) boundaries[14]. Local variables

[14]This refers to the interface between different binary modules, typically a user program and a system library. The ABI describes low-level details of that interface, for example the assignment of arguments and return values into registers or memory. In many systems, the ABI is expected to change rarely, so programs and libraries can be updated independently and still work together. This makes ABI changes undesirable, which is why this proposal aims to

which do not cross ABI boundaries are implicitly converted to use wide pointers. These wide pointers store bounds information alongside the original pointer.

There are also hardening efforts focusing on C++ codebases. For example, the libc++ hardening modes enable a number of assertions that aim to catch undefined behaviour in the library. The C++ Buffer Hardening proposal aims to extend this library hardening. The proposal will also introduce a programming model in which all pointer arithmetic is considered unsafe. Pointer arithmetic will have to be replaced with alternatives from the C++ library, for example `std::array`. The implementation of these alternatives in the hardened library will include bounds checks.

Successfully using bounds checking compiler features for a large codebase requires substantial effort. An example of this is refactoring the Linux kernel to use bounds checks for flexible arrays, as described in (Cook 2023).

There are also hardware-based mitigations for violations of spatial memory safety. For example, CHERI introduces *capabilities* to conventional Instruction Set Architectures. Capabilities combine a virtual address with metadata that describes its corresponding bounds and permissions. Capabilities cannot be forged, and can thus provide very strong guarantees. Arm has developed a prototype architecture that adapts CHERI, as well as a prototype SoC and development board, as part of the Arm Morello Program.

Of course, another approach to mitigating spatial memory safety vulnerabilities is using a language that has been designed with spatial memory safety in mind. Such languages make sure that all memory accesses are checked, either at compile-time or runtime. For example, the Rust programming language introduces bounds checks whenever the compiler cannot prove that an access is within bounds[15]. There are many other memory safe languages, with different characteristics. One example is JavaScript, a dynamically typed, usually JIT-compiled language. We'll discuss some of the issues that arise when implementing support for such a language in the next section.

2.9 JIT compiler vulnerabilities

Compiler correctness is obviously very important, as miscompilation creates buggy programs even when the source code has no bugs. What might be less obvious is that these bugs can have security implications. For example, they can introduce memory safety errors in languages that are otherwise memory safe. In some cases, a bug might leave most programs unaffected and not cause security issues in practice before it is detected and fixed. This is, of course, assuming that the bug has not been intentionally injected in the compiler.

Compiler bugs are an interesting source of security issues for just-in-time (JIT) compilers[16]. JIT compilation is often used in programs that receive source code as input during program execution, for example in web browsers, for executing JavaScript code included in web pages. In this context, the input to the JIT

minimise them.

[15] Rust also provides features that provide temporal memory safety and thread safety.

[16] JIT compilers compile code during execution of a program, as opposed to the more traditional compilation where code is compiled before the program is executed.

compiler comes from arbitrary websites and is therefore untrusted. Bugs in such JIT compilers can lead to compromise of the whole program (here, the browser) if a malicious input (e.g. coming from a malicious website) deliberately triggers miscompilation in order to break memory safety of the language being implemented.

For this section, we focus on JavaScript, which is a dynamically typed, memory safe language, but the concerns we discuss also apply to other languages that are compiled dynamically.

Without statically known types, in order to optimize JavaScript code, JavaScript engines resort to type profiling (Pizlo 2020), recording the types encountered while executing code. These types are then used during optimization, which speculates that the same types will be encountered in future runs of the code, and inserts checks to validate that these assumptions about types still hold. When a check fails, the optimized code is replaced by unoptimized code that can handle all types, a process known as deoptimization or on-stack replacement (OSR). Deoptimization makes sure that the state of the deoptimized function is recreated correctly for the point of execution where the type check failed.

For example, a function such as:

```
function foo(x, y) {
  return x + y;
}
```

will return a number when `x` and `y` are numbers, but a string when either is a string. An optimizing compiler can use the results of profiling to generate optimized code. For example, when both arguments are integers during profiling, it can generate code that looks like this in pseudocode:

```
foo:
  if x not integer, deoptimize
  if y not integer, deoptimize
  result = x + y
  if overflowed, deoptimize
  return result
```

You may be wondering how the type checks are implemented, and this is closely related to the representation of values in a JavaScript engine (Wingo 2011). In short, JavaScript engines use specific bit patterns to indicate whether a value should be interpreted as a pointer, or as an integer or floating-point value. For example, the V8 JavaScript engine uses the least significant bit to denote that a value is a pointer, otherwise it is a small integer (which needs to be shifted down to access its value). Pointers then point to objects that contain a hidden class member which is used for type checking.

In addition to the values for which typing information is gathered during profiling, optimizing JavaScript compilers propagate the profiled types to dependent values. For example if a value `x` is expected to be a string, and we check this assumption, then `x + 1` will also be a string (and no additional check is needed in this case). In addition to simple type propagation, they usually perform range analysis to determine as precise a range for a value as possible, which is useful for bounds check elimination.

Bounds check elimination (BCE) is a common optimization in languages that perform bounds checks on array accesses to ensure every accessed index is within the bounds of the array. BCE gets rid of bounds checks when they are proven to be redundant, e.g. when the array access uses a constant index that's known to be smaller than the length of the array. See here for details on how out-of-bounds array accesses behave in JavaScript.

Range analysis is a good example of an analysis where a JIT compiler bug can introduce a vulnerability. Incorrect range analysis results can be used by bounds check elimination to incorrectly eliminate bounds checks that should actually have been maintained in the optimized code. For example, for the following function:

```
function foo(x) {
  y = bar(x);
  var a = [0, 1, 2];
  return a[y];
}
```

If range analysis decides that the value of `y` is in the range `[0, 2]`, but in reality the value is in the range `[0, 3]`, the bounds check for the access `a[y]` can be eliminated incorrectly, assuming the access is in-bounds. (Glazunov 2021) lists a few examples of similar hypothetical vulnerabilities, along with examples of vulnerabilities of this type that affected widely-used JavaScript engines.

The type of bug described above provides an attacker with a limited read or write primitive, as a linear overflow of the array allocation occurs. The attacker can then build on this primitive to get to an arbitrary read/write primitive. As JIT compilers generate executable code at runtime, they often use memory that is writable and executable at the same time. Such memory is very useful to attackers, who can use an arbitrary write primitive to copy their payload into this code memory, and then jump to it. Writable and executable memory, therefore, makes JITs lucrative targets for attackers.

Bugs related to range analysis are just one of the common types of bugs encountered in a JavaScript engine. (Groß and Burnett 2022) lists some other common types of bugs that result in violations of temporal and spatial memory safety, as well as type safety, in JavaScript engines.

How can we defend against such vulnerabilities? There are several complementary approaches, for example:

1. Use fuzzing to discover compiler bugs. For JavaScript, a useful fuzzing tool is Fuzzilli.
2. Be more conservative when it comes to error-prone compiler optimizations such as bounds check elimination. For example, the V8 JavaScript engine has introduced hardening of bounds checks against typer bugs [17].
3. Instead of trying to prevent compiler (and other) bugs, assume they will be present and introduce mitigations that prevent attackers from building arbitrary read/write primitives on top of the initial limited primitives that bugs provide. For example, for 64-bit architectures, V8 implements a sandbox, built on top of pointer compression. With pointer compression,

[17] This naturally leads to attempts to bypass the hardening too (Fetiveau 2019).

pointers are represented by 32-bit indices off a base pointer instead of as full 64-bit values. By making sure that all pointers inside the sandbox (where the JavaScript heap is located) are compressed, and that compressed pointers always point inside the sandbox, a limited primitive that allows overwriting memory within the sandbox cannot be used to build an arbitrary read/write primitive by overwriting pointer values.

4. Preventing code memory from being executable and writable at the same time is also desirable. This is known as W^X. A naive implementation of W^X that simply switches memory permissions based on page tables temporarily is not enough to prevent attackers from writing to code memory (Song et al. 2015), when multiple threads are involved. A more effective solution would use a separate compilation process, which is the only process that has write access to the JIT's code memory. Alternatively, some architectures provide special features that can restrict page-based memory permissions from userspace, effectively allowing permissions to be different for different threads. Such features can also be of use in implementing W^X. For AArch64, this feature is called permission overlays.

In this section, we have discussed JIT compiler security and described JavaScript compiler bugs that lead to vulnerabilities. Although we haven't focused on the details of JavaScript exploitation, an interested reader could take a look at (saelo 2021b) and (saelo 2021a).

Chapter 3

Covert channels and side-channels

Side-channels and covert channels are communication channels between two entities in a system, where the entities should not be able to communicate that way.

A **covert channel** is a channel where both entities intend to communicate. A **side-channel** is a channel where one entity is the victim of an attack using the channel.

The difference between a covert channel and a side-channel is whether both entities intend to communicate. In a side-channel attack, the entity not intending to communicate is called the **victim**. The other entity is sometimes called the **spy**.

As we focus on attacks in this book, we'll mostly use the term side-channels in the rest of this chapter.

The next few sections describe a variety of side-channels. Each section focusses on leakage through a specific so-called micro-architectural aspect, such as execution time, cache state or branch predictor state.

3.1 Timing side-channels

An implementation of a cryptographic algorithm can leak information about the data it processes if its run time is influenced by the value of the processed data. Attacks making use of this are called timing attacks.

The main mitigation against such attacks consists of carefully implementing the algorithm such that the execution time remains independent of the processed data. This can be done by making sure that both:

a) The control flow, i.e. the trace of instructions executed, does not change depending on the processed data. This guarantees that every time the

algorithm runs, exactly the same sequence of instructions is executed, independent of the processed data.

b) The instructions used to implement the algorithm are from the subset of instructions for which the execution time is known to not depend on the data values it processes.

   For example, in the Arm architecture, the Armv8.4-A DIT extension guarantees that execution time is data-independent for a subset of the AArch64 instructions.

   By ensuring that the extension is enabled and only instructions in the subset are used, data-independent execution time is guaranteed.

At the moment, we do not know of a compiler implementation that actively helps to guarantee both (a) and (b).

Using compiler techniques to transform a function such that it respects property (a) is an active research area. (Wu et al. 2018) provides a method to convert a program such that it respects property (a), albeit by potentially introducing unsafe memory accesses. (Soares and Pereira 2021) improves on that result by not introducing unsafe memory accesses, albeit by potentially needing to change the interface of the transformed function.

Also discuss the techniques implemented in the Constatine compiler #172

A great reference giving practical advice on how to achieve (a), (b) and more security hardening properties specific for cryptographic kernels is found in (Pornin 2018).

As discussed in (Pornin 2018), when implementing cryptographic algorithms, you also need to keep cache side-channel attacks in mind, which are discussed in the section on cache side-channel attacks.

3.2 Cache side-channels

Caches are used in almost every computing system. They are small memories that are much faster than the main memory. They automatically keep the most frequently used data, so that the average memory access time improves.

Also discuss the Jasmin language and compiler 1 2 #213

When processes share a cache, various techniques exist to establish a covert communication channel. These let the processes communicate through memory accesses even when they do not share any memory location. We first describe how caches work before exploring these techniques.

3.2.1 Typical CPU cache architecture

There is a wide variety in CPU cache micro-architecture details, but the main characteristics that are important to set up a covert channel tend to be similar across most popular implementations.

Caches are small and much faster memories than the main memory that aim to keep a copy of the data at the most frequently accessed main memory addresses. The set of addresses that are used most frequently changes quickly over time as a program executes. Therefore, the addresses that are present in CPU caches

also evolve quickly over time. The content of the cache may change with every executed read or write instruction.

On every read and write instruction, the cache micro-architecture looks up if the data for the requested address happens to be present in the cache. If it is, the CPU can continue executing quickly; if not, dependent operations will have to wait until the data returns from the slower main memory. A typical access time is 3 to 5 CPU cycles for the fastest cache on a CPU versus hundreds of cycles for a main memory access. When data is present in the cache for a read or write, it is said to be a **cache hit**. Otherwise, it's called a **cache miss**.

Most systems have multiple levels of cache, each with a different trade-off between cache size and access time. Some typical characteristics might be:

- L1 (level 1) cache, 32KB in size, with an access time of 4 cycles.
- L2 cache, 256KB in size, with an access time of 10 cycles.
- L3 cache, 16MB in size, with an access time of 40 cycles.
- Main memory, gigabytes in size, with an access time of more than 100 cycles.

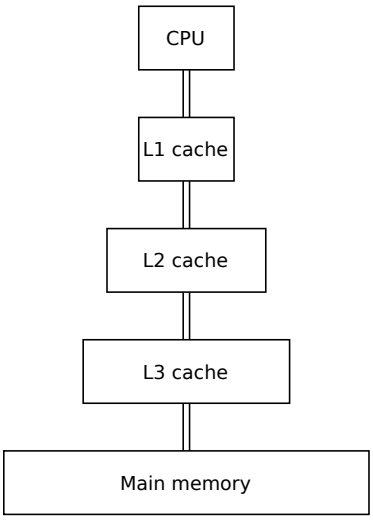


Illustration of cache levels in a typical system

If data is not already present in a cache layer, it is typically stored there after it has been fetched from a slower cache level or main memory. This is often a good decision to make as there's a high likelihood the same address will be accessed by the program soon after. This high likelihood is known as the principle of locality].

Data is stored and transferred between cache levels in blocks of aligned memory. Such a block is called a **cache block** or **cache line**. Typical sizes are 32, 64 or 128 bytes per cache line.

When data that wasn't previously in the cache needs to be stored in the cache, room has to be made for it by removing, or **evicting**, some other address/data from it. How that choice gets made is decided by the cache replacement policy].

Popular replacement algorithms are Least Recently Used (LRU), Random and pseudo-LRU. As the names suggest, LRU evicts the cache line that is least recently used; random picks a random cache line; and pseudo-LRU approximates choosing the least recently used line.

If a cache line can be stored in all locations available in the cache, the cache is **fully-associative**. Most caches are however not fully-associative, as it's too costly to implement. Instead, most caches are **set-associative**. In an N-way set-associative cache, a specific line can only be stored in one of N cache locations. For example, if a line can potentially be stored in one of 2 locations, the cache is said to be 2-way set-associative. If it can be stored in one of 4 locations, it's called 4-way set-associative, and so on. When an address can only be stored in one location in the cache, it is said to be **direct-mapped**, rather than 1-way set-associative. Typical organizations are direct-mapped, 2-way, 4-way, 8-way, 16-way or 32-way set-associative.

The set of cache locations that a particular cache line can be stored at is called a **cache set**.

3.2.1.1 Indexing in a set-associative cache

For some cache covert channels, it is essential to know exactly how a memory address maps to a specific cache set.

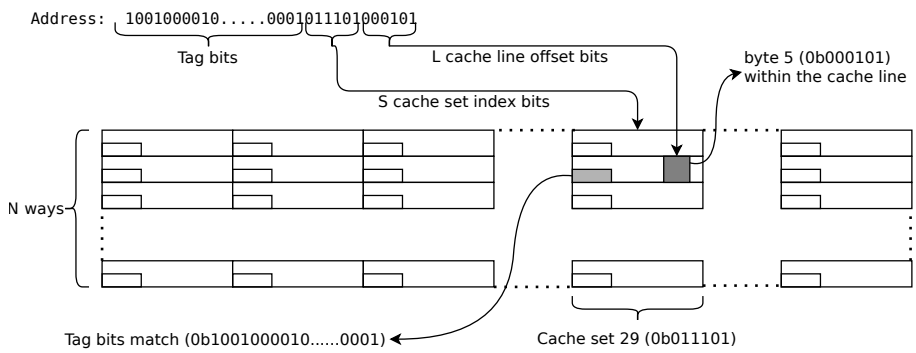


Figure 6: Illustration of indexing into a set-associative cache. In this example: $L = 6$ bits, hence the cache line size is $2^6 = 64$ bytes. $S = 5$ bits, so there are $2^5 = 32$ cache sets. N can be independent of address bits used to index the cache. If we assume $N = 12$ for a 12-way set-associative cache, the total cache size is $N * 2^L * 2^S = 12 * 64 * 32 = 24$KB.

Specific bits in the memory address are used for different cache indexing purposes, as illustrated in figure 6. The least-significant L bits, where 2^L is the cache line size, are used to compute an address's offset within a cache line. The next S bits, where 2^S is the number of cache sets, are used to determine which cache set an address maps to. The remaining top bits are "tag bits". They are stored alongside a line in the cache so later operations can detect which specific memory address is replicated in that cache line.

For direct-mapped and fully-associative caches, the mapping of an address to

cache locations also works as described above. In fully-associative caches the number of cache sets is 1, so S=0.

Also explain cache coherency cache coherency? #173

Also say something about TLBs and prefetching? #174

3.2.2 Operation of cache side-channels

Cache side-channels typically work by the spy determining whether a memory access was a cache hit or a cache miss. From that information, the spy may be able to deduce bits of data that only the victim should have access to.

Let's illustrate this by describing a few well-known cache side-channels:

3.2.2.1 Flush+Reload

In a so-called **Flush+Reload** attack(Yarom and Falkner 2014), the spy process shares memory with the victim process. The attack works in 3 steps:

1. The Flush step: The spy flushes a specific address from the cache.
2. The spy waits for some time to give the victim time to potentially access that address, resulting in bringing it back into the cache.
3. The Reload step: The spy accesses the address and measures the access time. A short access time means the address is in the cache; a long access time means it's not in the cache. In other words, a short access time means that in step 2 the victim accessed the address; a long access time means it did not access the address.

Should there be a more elaborate example with code that demonstrates in more detail how a flush+reload attack works? #175

Knowing if a victim accessed a specific address can leak sensitive information. Such as when accessing a specific array element depends on whether a specific bit is set in secret data. For example, (Yarom and Falkner 2014) demonstrates that a Flush+Reload attack can be used to leak GnuPG private keys.

3.2.2.2 Prime+Probe

In a **Prime+Probe** attack, there is no need for memory to be shared between victim and spy. The attack works in 3 steps:

1. The Prime step: The spy fills one or more cache sets with its data, for example, by accessing data that maps to those cache sets.
2. The spy waits for some time to let the victim potentially access data that maps to those same cache sets.
3. The Probe step: The spy accesses that same data as in the prime step. Measuring the time it takes to load the data, it can derive how many cache

lines the victim evicted from each cache set in step 2, and from that derive information about addresses the victim accessed.

(Osvik, Shamir, and Tromer 2005) first documented this technique in 2005 and demonstrates extracting AES keys in just a few milliseconds.

3.2.2.3 General schema for cache covert channels

An attentive reader may have noticed that the attacks described above follow a similar 3-step pattern. (Weber et al. 2021) describes this general pattern and uses it to automatically discover more side-channels that follow this 3-step pattern. They describe the general pattern as being:

1. An instruction sequence that resets the inner CPU state (**reset sequence**).
2. An instruction sequence that triggers a state change (**trigger sequence**).
3. An instruction sequence that leaks the inner state (**measurement sequence**).

Other cache-based side channel attacks following this general 3-step approach include: Flush+Flush(Gruss, Maurice, Wagner, et al. 2016), Flush+Prefetch(Gruss, Maurice, Fogh, et al. 2016), Evict+Reload(Percival 2005), Evict+Time(Osvik, Shamir, and Tromer 2005), Reload+Refresh(Briongos et al. 2020), Collide+Probe(Lipp et al. 2020), etc.

3.2.3 Mitigating cache side-channel attacks

As described in (Su and Zeng 2021), 3 conditions need to be met for a cache-based side-channel attack to succeed:

1. There is a mapping between a state change in the cache and sensitive information in the victim program.
2. The spy runs on a CPU that shares a cache level with the CPU the victim runs on.
3. The spy can infer a cache status change caused by the victim through its own cache status.

Mitigations against cache side-channel attacks can be categorized according to which of the 3 conditions above they aim to prevent from happening:

3.2.3.1 Mitigations de-correlating cache state change with sensitive information in the victim program

A typical example of when a cache state change could be correlated to sensitive information is when a program uses secret information to index into an array. An attacker could derive bits of the secret information by observing which cache line was fetched.

Especially in crypto kernels, indexing into an array using a secret value is generally avoided. An alternative mitigation is to always access all array indices, independent of the secret value, e.g. as done in commit 46fbe375 to the PuTTY project, which contains this comment:

```
* Side-channel considerations: the exponent is secret, so
* actually doing a single table lookup by using a chunk of
```

```
 * exponent bits as an array index would be an obvious leak of
 * secret information into the cache. So instead, in each
 * iteration, we read _all_ the table entries, and do a sequence
 * of mp_select operations to leave just the one we wanted in the
 * variable
```

3.2.3.2 Mitigations disallowing spy programs to share the cache with the victim program

If the victim and the spy do not share a common channel – in this case a cache level – then a side channel cannot be created.

One way to achieve this is to only allow one program to run at the same time, and when a context switch does happen, to clear all cache content. Obviously, this has a huge performance impact, especially in systems with multiple cores and with large caches. Therefore, a wide variety of mitigations have been proposed that aim to make attacks somewhat harder without losing too much system efficiency. (Mushtaq et al. 2020) and (Su and Zeng 2021) summarize dozens of proposals and implementations – too many to try to describe them all here.

One popular such mitigation is disabling cpu multithreading. For example, Azure suggests that users who run untrusted code should consider disabling cpu multithreading. The linux kernel's core scheduling documentation also states mutually untrusted code should not run on the same core concurrently. It implements a scheduler that takes into account which processes are mutually-trusting and only allows those to run simultaneously on the same core.

One could argue that site isolation as implemented in many web browsers is a mitigation that also falls into this category. Site isolation is described in more detail in its own section.

3.2.3.3 Mitigations disabling the spy program to infer a cache status change in the victim program through its own cache status

In some contexts, the resolution of the smallest time increment measurable by the spy program can be reduced so much that it becomes much harder to distinguish between a cache hit and a cache miss. Injecting noise and jitter into the timer also makes it harder to distinguish between a cache hit and cache miss. This is one of the mitigations in javascript engines against Spectre attacks. For more information see this v8 blog post or this Firefox documentation of the performance.now() method.

Note that this is not a perfect mitigation - there are often surprising ways that an attacker can get a fine-grained enough timer or use statistical methods to be able to detect the difference between a cache hit or miss. One extreme example is the NetSpectre attack (Schwarz et al. 2019) where the difference between cache hit and cache miss is measured over a network, by statistically analyzing delays on network packet responses. Furthermore, (Schwarz et al. 2017) demonstrates how to construct high-resolution timers in various indirect ways in all browsers that have removed explicit fine-grained timers.

Another possibility is to clear the cache between times when the victim runs and the spy runs. This is probably going to incur quite a bit of performance

overhead, and may also not always possible e.g. when victim and spy are running at the same time on 2 CPUs sharing a cache level.

3.3 Branch-predictor based side-channels

3.3.1 Branch predictors

Most CPUs implement one or more instruction pipelines. In an instruction pipeline, the next instruction is started before the previous instruction has finished executing. When the previous instruction is a branch instruction, the next instruction that needs to be executed is only known when that branch instruction completes. However, waiting for the branch instruction to finish before starting the next instruction leads to a big performance loss.[1] Therefore, most CPUs predict which instruction needs to be executed after a branch, before the branch instruction has completed. Correctly and quickly predicting the instruction after a branch instruction is so important for performance that most CPUs have multiple branch predictors, such as:

- A predictor of the outcome of a conditional branch: taken or not taken. The prediction is typically history-based, i.e. based on the outcome of this and other branches in the recent past.
- A predictor of the target of a taken branch, i.e. the address of the next instruction after a taken branch.
- A predictor that is specialized to predict the next instruction after a function return instruction.

3.3.2 Side-channels through branch predictors

A number of attacks have been described over the past few years. The following sections list a few examples, categorized per branch predictor component they target.

3.3.2.1 Conditional branch direction predictor side-channel attacks

Two examples are BranchScope (Evtyushkin et al. 2018) and BlueThunder (Huo et al. 2019). These attacks infer whether a branch is taken or not taken in a victim process. They do so by carefully making sure that a branch in the spy process uses the same branch predictor entry as the targeted branch in the victim process. By measuring whether the branch in the spy process gets predicted correctly, one can derive whether the branch in the victim process was taken or not.

This can be thought of as somewhat akin to the Prime+Probe cache-based side channel attacks.

When the outcome of a branch depends on a bit in a secret key, this can enable an attacker to derive the value of the secret key. These papers demonstrate

[1] Over time, new CPU designs tend to support having more instructions in flight. (Eyerman et al. 2009, sec. 4.2.3) suggests that branch prediction accuracy has to grow more than linearly when the number of pipelines, or the depth of the pipeline grows. Therefore, there is a constant push to increase the accuracy of branch predictors.

deriving the secret key from implementations of specific cryptographic kernels. It can also be used to break ASLR.

3.3.2.2 Branch target predictor side-channel attacks

Two examples are SBPA (Aciiçmez, Kaya Koç, and Seifert 2007) and BranchShadow (Lee et al. 2017). These earlier attacks are based on making a branch in the spy process alias in the Branch Target Buffer (BTB) with a targeted branch in the victim process. They use methods such as timing difference, last branch records, instruction traces or performance counters to measure whether the branch in the spy process caused a specific state change in the BTB.

3.3.2.3 Return address predictor side-channel attacks

One example is Hyper-Channel (Bulygin 2008). In this case, a spy process invokes N calls to fill up the return stack predictor. Then it lets the victim process execute. Then, the spy process can measure how many of its return stack entries have been removed from the Return Stack Buffer (RSB), by measuring the number of N returns that get mis-predicted. If the number of calls in the victim process is dependent on secret information, this could leak it.

The papers referred to above contain detailed explanations of how they set up the attack. All of these attacks use a general 3-step approach, similar to cache side channels:

1. An instruction sequence that resets the branch predictor state (*reset sequence*), run by the spy process.
2. An instruction sequence that triggers a branch predictor state change (*trigger sequence*), run by the victim process.
3. An instruction sequence that leaks the branch predictor state (*measurement sequence*), run by the spy process.

3.3.3 Mitigations

Describe the mitigations proposed against these side-channel attacks. #203

3.4 Resource contention channels

3.5 Channels making use of aliasing in other predictors

Should we also discuss more "covert" channels here such as power analysis, etc? #176

3.6 Transient execution attacks

3.6.1 Transient execution

3.6.1.1 Speculative execution

CPUs execute sequences of instructions. There are often dependencies between instructions in the sequence. That means that the outcome of one instruction influences the execution of a later instruction.

Apart from the smallest micro-controllers, all CPUs execute multiple instructions in parallel. Sometimes even multiple hundreds of them at the same time, all in various stages of execution. Instructions start executing while potentially hundreds of previous instructions haven't produced their results yet. How can a CPU achieve this when the output of a previous instruction, which might not have fully executed yet, and hence whose output may not yet be ready, may affect the execution of that later instruction? In other words, there may be a **dependency** between an instruction that has not finished yet and a later instruction that the CPU also already started executing.

There are various kinds of dependencies. One kind is **control dependencies**, where whether the later instruction should be executed at all is dependent on the outcome of the earlier instruction. Other kinds are **true data dependencies**, **anti-dependencies** and **output dependencies**. More details about these kinds of dependencies can be found on the wikipedia page about them.

CPUs overcome parallel execution limitations imposed by dependencies by making massive numbers of **predictions**. For example, most CPUs predict whether conditional branches are taken or not, which is making a prediction on control dependencies. Another example is a CPU making a prediction on whether a load accesses the same memory address as a preceding store. If they do not access the same memory locations, the load can run in parallel with the store, as there is no data dependency between them. If they do access overlapping memory locations, there is a dependency and the store should complete before the load can start executing.

Starting to execute later instructions before all of their dependencies have been resolved, based on the predictions, is called **speculation**.

Let's illustrate that with an example. The following C code

```c
long abs(long a) {
  if (a >= 0)
    return a;
  else
    return -a;
}
```

can be translated to the following AArch64 assembly code:

```
        cmp     x0, #0
        b.ge    Lbb2
Lbb1:
        neg     x0, x0
```

```
Lbb2:
        ret
```

The `b.ge` instruction is a conditional branch instruction. It computes whether the next instruction should be the one immediately after, or the one pointed to by label `Lbb2`. In case it's the instruction immediately after, the branch is said to not be taken. Instead, if it's the instruction pointed to be label `Lbb2`, the branch is said to be taken. When the condition `.ge` (greater or equal) is true, the branch is taken. That condition is defined or set by the previous instruction, the `cmp x0, #0` instruction, which compares the value in register `x0` with 0. Therefore, there is a dependency between the `cmp` instruction and the `b.ge` instruction. To overcome this dependency, and be able to execute the `cmp`, `b.ge` and potentially more instructions in parallel, the CPU predicts the outcome of the branch instruction. In other words, it predicts whether the branch is taken or not. The CPU will pick up either the `neg` or the `ret` instruction to start executing next. This is called *speculation*, as the CPU *speculatively executes* either instruction `neg`, or `ret`.

Show a second example of cpu speculation that is not based on branch prediction. #177

Of course, as with all predictions, the CPU gets the prediction wrong from time to time. In that case, all changes to the system state that affect the correct execution of the program need to be undone. In the above example, if the branch should have been taken, but the CPU predicted it to not be taken, the `neg` instruction is executed incorrectly and changes the value in register `x0`. After discovering the branch was mis-predicted, the CPU would have to restore the correct, non-negated, value in register `x0`.

Any instructions that are executed under so-called **mis-speculation**, are called **transient instructions**.[2]

The paragraph above says "*the system state that affects the correct execution of the program, needs to be undone*". There is a lot of system state that does not affect the correct execution of a program. And the changes to such system state by transient instructions is often not undone.

For example, a transient load instruction can fetch a value into the cache that was not there before. By bringing that value in the cache, it could have evicted another value from the cache. Whether a value is present in the cache does not influence the correct execution of a program; it merely influences its execution speed. Therefore, the effect of transient execution on the content of the cache is typically not undone when detecting mis-speculation.

Sometimes, it is said that the **architectural effects** of transient instructions need to be undone, but the **micro-architectural effects** do not need to be undone.

The above explanation describes architectural effects as changes in system state

[2]Transient instructions caused by incorrect branch-direction prediction have also been called **wrong-path instructions** Mutlu et al. (2004)

that need to be undone after detecting mis-speculation. In reality, most systems will implement techniques that keep all state changes in micro-architectural buffers until it is clear that all predictions made to execute that instruction were correct. At that point the micro-architectural state is **committed** to become architectural state. In that way, mis-predictions naturally do not affect architectural state.

Could we find a good reference that explains micro-architectural versus architectural state in more detail? Is "Computer Architecture: A Quantitative Approach" the best reference available?

Faulting instructions are instructions that generate an exception at runtime. Many instructions can generate an exception, and are hence **potentially faulting**. For example most load and store instructions generate an exception when the accessed address is not mapped. Since so many instructions can generate an exception, processors typically speculate that they do not generate an exception to enable more parallel execution.

When an instruction faults, the execution typically continues at another location. Any instructions later in the instruction stream which are speculatively executed before the fault is detected are also called **transient instructions**.

There is a kind of control dependency between every potentially-faulting instruction and the next one, as the next instruction to be executed depends on whether the instruction generates an exception or not. We call out this dependency separately here as the transient execution attacks we'll describe next get classified based on whether they make use of transient instructions after a misprediction, or transient instructions after a faulting instruction.

3.6.2 Transient Execution Attacks

Transient execution attacks are a category of side-channel attacks that use the micro-architectural side-effects of transient execution as a side channel.

The publication of the Spectre (Kocher et al. 2019) and Meltdown (Lipp et al. 2018) attacks in 2018 started a period in which a large number of transient attacks were discovered and published. Most of them were given specific names, such as ZombieLoad, NetSpectre, LVI, Straight-line Speculation, etc. New variants continue to be published regularly.

Covering each one of them in detail here would make the book overly lengthy, and may not necessarily help much with gaining a better insight in the common characteristics of transient attacks. Therefore, we'll try to put them into a few categories and describe the characteristics of each category.

Decide whether it's useful to talk about alternative categorizations of transient execution attacks, and if so, do add content. Consider pointing to https://github.com/MattPD/cpplinks/blob/master/comparch.micro.channels.md

The categorization below is based on one proposed in (Bulck et al., n.d.). There are alternative categorizations. (Bulck et al., n.d.) defines 4 big classes of transient side-channel attack categories, based on whether:

1. The transient execution happens because of a misprediction, or a faulting instruction.

2. The attacker actively steers data or control flow of the transient execution or not.

This gives the following 4 categories:

	Steering of transient execution by attacker?	
	No (Leakage)	Yes (Injection)
Misprediction	Branch-predictor based side-channels	Spectre-style attacks
Faulting	Meltdown-style attacks	LVI-style attacks

3.6.2.1 Branch predictor-based side-channel attacks

We discussed this category already in the section on Side-channels through branch predictors.

3.6.2.2 Spectre-style attacks

Add a description of Spectre-style attacks such as Spectre-PHT, Spectre-BTB, Spectre-RSB, Spectre-STL, SpectreV1, SpectreV2, SpectreV3, SpectreV4, NetSpectre. #178

3.6.2.3 Meltdown-style attacks

Add a description of Meltdown-style attacks such as Meltdown, Foreshadow, LazyFP, Fallout, ZombieLoad, RIDL. #178

3.6.2.4 LVI-style attacks

Add a description of LVI-style attacks. #178

3.6.3 Mitigations against transient execution attacks

3.6.3.1 Site isolation

Write section on site isolation as a SpectreV1 mitigation #179

3.7 Physical access side-channel attacks

Chapter 4

Supply chain attacks

A software *supply chain attack* occurs when an attacker interferes with the software development or distribution processes with the intention to impact users of that software.

Supply chain attacks and their possible mitigations are not specific to compilers. However, compilers are an attractive target for attack because they are widely deployed to developers, in continuous integration systems and as JITs. Also, an infected compiler has the possibility to make a much larger impact if it can silently spread the infection to other software created with or run using it.

This chapter explores the history of supply chain attacks that involve compilers and what can be done to prevent them.

4.1 History of supply chain attacks

As far back as 1974 Karger & Schell theorized about an attack on the Multics operating system via the PL/I compiler (Paul A. and Roger R. 1974). In this attack, a trap door is inserted into the compiler, which then injects malicious code into generated object code. Furthermore, the trap door could be designed to reinsert itself into the compiler binary so that future compilers are silently infected without needing changes to their source code. This attack method was subsequently popularized by Ken Thompson in his 1984 ACM Turing Award acceptance speech *Reflections on Trusting Trust* (Thompson 1984).

If these cases seem far-fetched then consider that there have been several real examples of supply chain attacks on development tools.

Induc is a family of viruses that infects a pre-compiled library in the Delphi toolchain with malicious code (Gostev 2009). When Delphi compiles a project the malicious library is included into the resulting executable, thus enabling the virus to spread. The virus was first detected in 2009 and was circulating undetected for at least a year beforehand. Several popular applications are known to have been infected, including a chat client and a media player. Overall, in excess of a hundred thousand infected computers were detected world-wide by anti-virus solutions.

XcodeGhost is the name given to malware first detected in 2015 that infected thousands of iOS applications (Cox 2015). The source of the infection was tracked down to a trojanized version of Xcode tools. The malware exists in an extra object file within the Xcode tools and is silently linked into each application as it is built. File sharing sites were used to spread the trojanized Xcode tools to unwitting developers.

A trojanized linker was found to be involved in a supply chain attack discovered in 2017 named ShadowPad (Greenberg 2019). Some instances of the attack were perpetrated using a trojanized Visual Studio linker that silently incorporates a malicious library into applications as they are built. Related attacks named CCleaner and ShadowHammer used the same approach of a trojanized linker to infect built applications. Infected applications from these attacks were distributed to millions of users world-wide.

These cases highlight that attacks on compilers, and especially linkers and libraries, are a viable route to silently infect many other applications, and there is no doubt that there will be more such attacks in the future. Let us now explore what we can do about these.

Explain how these vulnerabilities arise and how to mitigate them. #180

Chapter 5

Compiler introduced security vulnerabilities

Security vulnerabilities introduced by compilers have a long history. Thompson (Thompson 1984) provides one of the oldest and most popular examples in this area. In his paper, he talks about a compiler that can detect when it is compiling the login program and can insert a backdoor so that he can use the system as any user. However most common cases are where involuntary security vulnerabilities are added in the generated binary by the compiler.

Explain how code that results in undefined behaviour can often work as the programmer expected until some optimisation is applied, and perhaps even talk a bit about why compilers rely on the absence of undefined behaviour in ways that appear aggressive in some occasions. #202

When discussing compiler introduced security vulnerabilities, undefined behavior plays a major role. Its implications were thoroughly discussed by various works such as (Wang et al. 2012) (D'Silva, Payer, and Song 2015) (Du, Wu, and Mao, n.d.). By reading the works of these authors, one can see that even projects that went through careful testing, such as Linux, FreeBSD or PostgreSQL, could not escape from this class of vulnerabilities. To better understand them, this chapter contains several examples of such vulnerabilities, their implications and how they got fixed.

The first example is a 15 years old vulnerability that affected the random number generator (RNG) in Mac OS X (Wang 2015). At some point in the past, this vulnerability affected all *BSD operating systems, as they have a common ancestor with Mac OS.

In the random number generator of the system, more specifically in `srandomdev(3)`, we can spot the following piece of code used in the seeding logic:

```
struct timeval tv;
```

```
unsigned long junk;

gettimeofday(&tv, NULL);
srandom((getpid() << 16) ^ tv.tv_sec ^ tv.tv_usec ^ junk);
```

For generating a seed for the RNG, the code uses the current time and an uninitialized value from the stack, i.e. `junk`. This triggers undefined behavior as the C standard has no clear semantics for uninitialized loads. Because of that, there was a huge difference in the generated assembly code for two different Mac OS X releases.

In Mac OS X 10.6 the generated code looked like this:

```
leaq    0xe0(%rbp),%rdi
xorl    %esi,%esi
callq   0x001422ca      ; symbol stub for: _gettimeofday
callq   0x00142270      ; symbol stub for: _getpid
movq    0xe0(%rbp),%rdx
movl    0xe8(%rbp),%edi
xorl    %edx,%edi
shll    $0x10,%eax
xorl    %eax,%edi
xorl    %ebx,%edi
callq   0x00142d68      ; symbol stub for: _srandom
```

While for Mac OS X 10.7 the code looked like this:

```
leaq    0xd8(%rbp),%rdi
xorl    %esi,%esi
callq   0x000a427e      ; symbol stub for: _gettimeofday
callq   0x000a3882      ; symbol stub for: _getpid
callq   0x000a4752      ; symbol stub for: _srandom
```

In the shorter version of the generated assembly code, the compiler dropped the whole argument of `srandom` as an optimization. While the optimised code respects the standard, it leaves room for an attacker to exploit the system because the seed of the RNG can now be predicted.

In the meantime, this problem has been resolved in FreeBSD ("FreeBSD Mitigation for Srandom Undefined Behavior" 2012) and OpenBSD ("OpenBSD Mitigation for Srandom Undefined Behavior" 2002).

Current solutions for detecting this class of vulnerabilities include LLVM's MemorySanitizer and Valgrind.

The next example covers a new type of undefined behavior that can easily introduce security vulnerabilities. This time we talk about dereferencing NULL pointers and what might go wrong with this operation. The following piece of code is taken from Linux and introduces a vulnerability by dereferencing the `tun` pointer before it checks that the pointer is valid:

```
unsigned int
tun_chr_poll(struct file *file, poll_table * wait)
{
  struct tun_file *tfile = file->private_data;
```

```
  struct tun_struct *tun = __tun_get(tfile);
  struct sock *sk = tun->sk;
  if (!tun)
    return POLLERR;
  ...
}
```

Normally, this would cause a crash in the kernel or the function would return POLLERR if address 0 was mapped in the address space. However the compiler assumes that `tun` is a valid pointer when the execution reaches the if statement. This happens because it saw an earlier dereference just before the if statement. In this situation, the check is considered redundant and deleted from the final binary. This allows an attacker to continue executing code from `tun_chr_poll` when address 0 is mapped.

To mitigate against this situation, GCC developers added a flag called `-fno-delete-null-pointer-checks` that Linux integrated in its compiler configuration.

Linux was not the only project that suffered from this problem. Chromium ("Issue 3782: V8 Is Not -Fsanitize=null Clean" 2014) and Mozilla ("GCC6 - TB Crashes Due to Removed Null Pointer Checks for "This"" 2016) had problems in the past with this.

There are also cases of security vulnerabilities that are not introduced by undefined behavior, the following piece of code is such an example. This was taken from the Linux kernel. Because the compiler sees that the pointer hash is never used after this point, it decides to delete the memset operation. We call this dead store optimization (DSO). This has serious security implications because the intention of the programmer was to delete the `hash` information from memory.

```
static void extract_buf(struct entropy_store *r, __u8 *out) {
  ...
  - memset(&hash, 0, sizeof(hash));
  + memzero_explicit(&hash, sizeof(hash));
}
```

The solution Linux came with was to add a new function called `memzero_explicit` which under the hood looks like this:

```
void memzero_explicit(void *s, size_t count)
{
  memset(s, 0, count);
  OPTIMIZER_HIDE_VAR(s);
}
```

It still uses `memset` to delete the associated security sensitive data, but it also tries to eliminate the risk of DSO by using the OPTIMIZER_HIDE_VAR macro. This, however, is not enough to fully eliminate dead stores ("Lib: Memzero_explicit: Use Barrier Instead of OPTIMIZER_HIDE_VAR" 2015). In case of using LTO, the buffer `s` is still vulnerable. For this reason, Linux maintainers added a further hardening mechanism by using a compiler barrier instead:

```
void memzero_explicit(void *s, size_t count)
{
  memset(s, 0, count);
  - OPTIMIZER_HIDE_VAR(s);
  + barrier();
}
```

There is still room for improvement regarding the introduced barrier ("Lib: Make Memzero_explicit More Robust Against Dead Store Elimination" 2015). If the content of the buffer is present in registers, then the compiler blindly proves again that the DSO can be triggered and the `memset` will be again deleted. To mitigate against this, the following patch was proposed:

```
+ #define barrier_data(ptr) \
+  __asm__ __volatile__("": :"r"(ptr) :"memory")

void memzero_explicit(void *s, size_t count)
{
  memset(s, 0, count);
  - barrier();
  + barrier_data(s);
}
```

In this patch we create a new barrier that will be guaranted to put the content of the buffer in memory so that DSO can take no further effect.

Similar efforts were conducted in other projects such as OpenSSL ("Lib: Make Memzero_explicit More Robust Against Dead Store Elimination" 2016). The approach OpenSSL used is rather different but it achieves the same end goal, i.e. eliminating the effect of DSO. By making `memset_func` a volatile pointer to the actual implementation of `memset`, the compiler is forced to dereference the pointer to get to the actual `memset`, thus eliminating the risk of optimizing it out.

The C23 committee decided to tackle this problem from another angle, i.e. by adding a library function called `memset_explicit` ("Memset_explicit" 2021). This function requires the compiler to not optimize the memory overwrite away. However it is not trivial to implement such a functionality, as GNU presents in ("10.604 Memset_explicit," n.d.). The information may be present somewhere in the machine, even if it was erased from memory.

Chapter 6

Underhanded code

Underhanded code is code that looks like it is doing one thing but actually does something else. It usually refers to code nefariously written in this way to sneak malicious behavior not obviously visible on code inspection. However the same applies also when the underhanded behavior has been added by mistake - in fact the ideal attack would also give the attacker the possibility to say it was an honest mistake.

(Wheeler 2020) provides a more systematic view of underhanded code than is given here. This chapter presents some examples to give an overview of the attack and how a compiler can (and can't) help, although this probably doesn't classify as "low level" software security.

6.1 Assignment and equality confusion

Probably the most classic example of underhanded code in C-style languages is code that takes advantage of the fact that the assignment (`=`) and the equality operator (`==`) look very similar.

```
void somefunction(UserPermissionLevel permission_level) {

   if (permission_level = LEVEL_ADMIN) {
       do_admin_action();
       return;
   }
   report_access_violation();
}
```

Here `permission_level` is not compared to `LEVEL_ADMIN`, but rather assigned that value. Meaning that if `LEVEL_ADMIN` is a non-zero value the if condition will evaluate to true.

Both Clang and GCC detects this example with warning option `-Wparentheses`. Adding a pair of parentheses around the assignment silences that warning. Those extra parentheses would probably draw a reviewer's attention in this specific

example but had the condition been more complex with multiple subexpressions combined with `||` and `&&` the parentheses had looked normal.

This could be written off as a language design problem. Traditionally Python has not supported using assignment as expressions, effectively sidestepping this particular problem. When assignment expressions were added to the language (Angelico, Peters, and Rossum 2018) (in python 3.8) the more visually distinct "walrus operator" `:=` was used to avoid this risk of confusing with the comparison operator. Even in cases where it is an existing language the compiler can provide options to forbid risky constructs for those who want and can opt in to it, essentially creating a new language that is a subset of the original language.

6.2 goto fail

The "goto-fail" bug, officially known as CVE-2014-1266, caused Apple devices to not correctly validate certificates in TLS connections. It effectively disables the verification the function is supposed to do, and is easy to miss in code review. This was most likely a mistake that was not added intentionally, but still has the characteristics of underhanded code.

The bug was caused by a duplicated "goto fail" line, which was indented making it look like it was guarded by previous if clause, when it in fact always was executed. Since the `err` variable was set to 0 (i.e no error) inside the if, the function would always return 0.

```
OSStatus
SSLVerifySignedServerKeyExchange(...)
{
    OSStatus err;
    ...

    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto fail;
        goto fail;
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto fail;
    ...

fail:
    return err;
}
```

At the time of the bug they didn't but nowadays both GCC and Clang have a `-Wmisleading-indentation` option that detects this kind problem.

Also automatic code formatting tools such as clang-format would help finding this issue as it will fix the misleading indentation.

6.3 Trojan Source

"Trojan Source" attacks described by (Boucher and Anderson 2023) is another way underhanded code could be achieved by doing something that makes the editor (or other thing that displays code to the user) render the code in a different way than the compiler parses it.

This is done by using unicode features. Support for bidirectional text is one such feature, where it has special characters to mark regions of text to be right-to-left or left-to-right to allow mixing e.g Arabic and English in the same document. If these are used in cunning ways they can make the text render in a way that makes a word look like it is *inside* a comment but to the compiler, which parses it in the order it appears in the file, sees it is *after* the comment.

GCC provides `-Wbidi-chars` to detect usage of writing direction markers. Clang compiler doesn't provide such warning, but has a check in clang-tidy

Another variant of "trojan source" is usage of homoglyphs, different characters that looks similar or even identical (depending on font). As an example, unicode contains the character "Division Slash" (U+2215) which is different from the ordinary ascii slash "Solidus" (U+002F).

```
/* Important comment */
check_permissions();
/* Another comment */
```

If the final slash on the first line isn't a slash but another similarly looking character that the compiler doesn't understand as end of comment it means that the whole snippet is commented out.

Chapter 7

Physical attacks

This chapter should probably be moved under section 'Physical access side-channel attacks' higher-up #181

7.1 Overview

There are many types of physical attacks – these attack methods focus on one or multiple physical properties of systems (e.g. CPU, GPU, crypto hardware), and can either be

- Passive – just monitoring physical quantities (e.g. side channel information leakage), or
- Active – modification of physical quantities, for example, by
  - changing the operation conditions of the system so that the circuit operates outside its specifications (e.g. by changing temperature, or by applying glitches to supply voltage/clock source)
  - injecting faults to the system (e.g. altering the electrical state of the system using Electromagnetic pulse injection, or laser beam)
  - physically modifying the system/chip

In the rest of this section, we will focus on a subset of physical attacks:

- Side channel information leakage
- Physical attack using glitches

These two forms of attacks can be carried out using low cost hardware, and have been widely demonstrated by researchers on SoCs or microcontrollers developed for IoT (Internet-of-Things) applications.

7.2 Physical access side-channel attacks

If an attacker has physical access to a device, even without debug access, the attacker can collect side channel information about the program execution on a

processor. If the processor is used to handle cryptographic operations, the side channel information can be used to deduce the crypto key(s) or the data being processed. Please note that some forms of physical attacks (e.g. fault injection attacks like rowhammer and voltjockey) do not require physical access, but those attacks are not covered in this section.

7.2.1 How is information leaked?

The most common physical access side channel attack method is to capture the voltage or current consumption of the device during its operation. Every time a flip-flop toggles, the switching activity results in a small current spike. Even though there is capacitance on the power distribution connections inside the chip, the toggling of registers (composed of flip-flops) still results in variations in the power supply current, which can be observed easily. Because the connections for delivering power (at the power supply, printed circuit board, on chip packages as well as on the silicon dies) also contain resistance, the variation of electrical current in the chip's power supply also results in variations in the power supply voltage. Again, this can be observed easily if the attacker has physical access to the device. If an attacker has access to data acquisition equipment that can record the current/voltage/power patterns, he/she can record the "power signature" for different crypto operations, including the power signature using different data inputs. By applying analysis techniques like differential power analysis, the attacker can extract the information being processed. One additional form of side-channel leakage is electro-magnetic radiation. Because the processor's clock frequency is usually in the radio frequency (RF) range, the wires on the die and the tracks on the PCB become small antennas, and the ripples in the processor's voltage/current results in radio frequency signals. Although the RF power radiated can be tiny, it still means that an attacker can observe the side-channel leakage if he/she is in close proximity from the device and has the right equipment to amplify and record the RF power signature. However, the risk of such attack can be reduced by reducing the radiation energy level using:

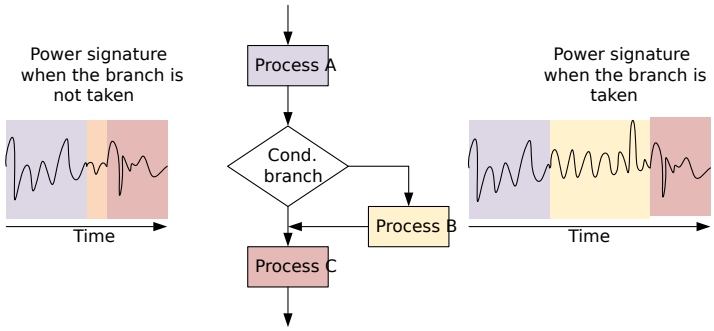
- Shielding around the device, including ground plate on the circuit board.
- Coupling capacitors on power supply tracks on the printed circuit board.

Generally, such an attack requires knowledge of radio circuit techniques and the result can be affected by other factors. For example, in normal environments there are many other source of RF noises that affects the accuracy of signal measurement. In a "noisy" environment, the RF signals from various wireless communication gadgets nearby might drown out the signals from the device being monitored.

7.2.2 Side channel leakage at instruction level

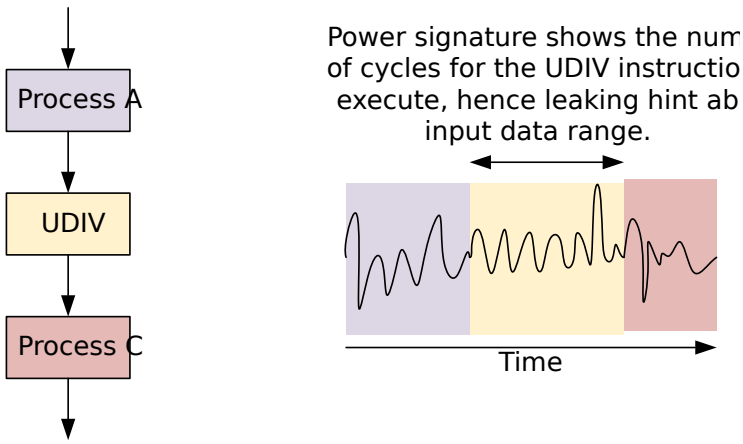
Instruction executions can result in various forms of side channel leakage:

Cycle timing resulting from conditional branch – A code sequence containing a conditional branch could result in observable side channel leakage. For example, if the power signatures of several code segments are easily recognizable (process A, B and C in the following diagram), it is possible to detect if the conditional branch was taken or not.



leakage of conditional branch

Cycle timing resulting from specific data values – The execution cycles of some instructions can be dependent on the values of input data, resulting in timing side-channel leakage. E.g., the integer divide instruction in Arm Cortex-M processors.



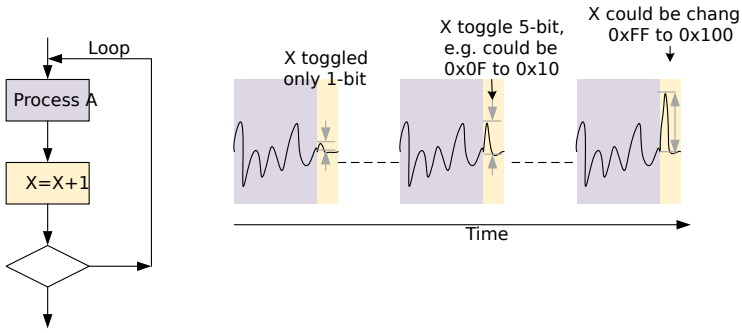
leakage of execution cycle

Power variation due to value changes – The power spikes in the power signature are often dependent on a combination of how many bits are set and how many bits have toggled in the register(s) — the so-called Hamming weight and Hamming distance, so the amplitude of the spike could be used to guess the register value in that clock cycle. The power spikes can be caused by a combination of

- Logic switching due to the operations of an instruction (e.g. power consumed by a single cycle multiplier can be much higher than the power used by a Boolean logic function), and
- Logic switching due to changes in data values in the register bank and data paths.

The switching activities are dependent on preceding and next operations. If the power signature of the codes around a specific instruction is recognizable, then

the data value being process could be guessed.



leakage of number of bit toggled

In some SoC or microcontroller implementations, the power spike effect of the operations can be much higher than the effect of data value changes in the register banks. In such case the program execution flow can be observed, and as a result, might also indirectly leak information about the data that it is processing.

7.2.3 Countermeasures

For normal embedded devices that don't have physical protection features, there is a much higher chance that power/voltage/radiation side channels can result in information leakage. However, some aspects of timing signature leakage could be reduced:

- Using data processing instruction with data independent timing for cryptographic operations. In recent Arm architectures (including Armv8-A and Armv8-M), some instructions are architecturally defined as DIT (Data Independent Timing).
- For conditional branches where the condition is dependent on secret data, use table branch instead might help reduce timing base leakage (both paths result in a branch). It is not necessary to replace all conditional branch. For example, many loop counters in crypto operation can be independent to the crypto key or input data values, so there is no need to change those loops.

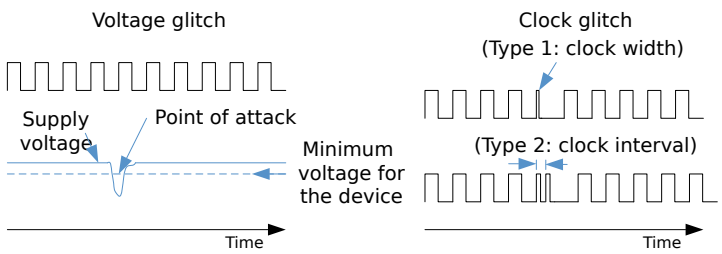
There is overlap with section timing-side-channels. How to best consolidate that? #182

There are additional software techniques to mitigate power leakage. One of the most well-known techniques is masking (e.g. Boolean, multiplicative, affine). When applying software mitigation, software developers need to check that optimizations carried out by compilers (C/C++) do not impact the mitigation, as compilers can be very smart and undo the masking in order to perform faster operations (or reducing code size).

7.3 Fault injection attacks

7.3.1 Common forms of Fault injection attacks

If an attacker has physical access to a device, they can also choose to use physical attacks to modify the behavior of the software, for example, prevent the software from setting up certain security features during the device's initialization sequence. The two most common forms of such attacks are voltage glitching and clock glitching.



common fault injection attacks

- Voltage glitch attack
  - Using a programmable power supply that can switch the voltage level rapidly, it is possible to reduce/increase the power supply voltage of a chip at specific clock cycle of the software execution. In some case, a precise voltage drop can cause a processor to "skip" an instruction, for example, the write to memory or a hardware register might not be taken. Or if a write has taken place, the actual write value used could be changed by the voltage glitch.
- Clock glitch attack
  - Using a clock switching circuit, it is possible to reduce the width of a clock pulse, or the interval between two clock pulses so that some of the hardware registers are not updated correctly at certain clock edge(s). Similar to voltage glitch, this can make the hardware seems to be skipping an instruction.

Such voltage/clock glitch attack could affect multiple parts in the processors, but sometimes the impact might not lead to any visible error in the operation, leaving the only effect that the processor skipping a memory/register write, or writing an incorrect value. Potentially, a glitch attack could result in other observable effect (e.g. register reset, bit toggle). The analysis of fault injection methods (and their physical effect) and the observable effects at the program or instruction execution level are often referred to as fault models, where one can say that a specific fault injection behaves as an instruction skip, etc. More details about the concept of fault models can be found in the paper "Fault Attacks on Secure Embedded Software: Threats, Design and Evaluation", where a good illustration of the concept is shown in figure 1 of that paper.

Make the above reference to a paper use bibtex. #159

Using glitching methods, there are several common ways of attacking a system. For example:

- Skipping an instruction during setup sequence for security features – e.g. skipping the write to the MPU (Memory Protection Unit)/Security Attribution Unit (SAU) so that the MPU/SAU is not enabled.
- Skipping an instruction after a security authentication that branch to an error handling code. As the branch is not taken, the code can continue to operate even a security authentication has failed.
- Causing an incorrect value to be written in a memory or hardware. E.g. When writing a crypto key to a crypto accelerator, forcing the key value written to be zero (caused to low voltage on bus hardware).

Example: Attack on TrustZone for Armv8-M

There are other forms of physical attacks, but most of them requires significant effort or cost (e.g. cut open the chip package can carry out fault injection or readout secret data on chip).

7.3.2 Countermeasures

Ideally, system designers can use hardware (SoCs or microcontroller) that support protection against fault injection. For example, a hardware circuit can include redundancy logic (spatial and temporal). In addition, software developers can make such attack harder by adding checks after the write operations. When applying software mitigation, software developers need to check that optimizations carried out by compilers (C/C++) do not impact the mitigation.

Chapter 8

Other security topics relevant for compiler developers

Write chapter with other security topics.

Appendix: contribution guidelines

If you'd like to start contributing to this book: please do, we're looking forward to your contributions!

The project lives on github at https://github.com/llsoftsec/llsoftsecbook. We also have a Discord server where you can have an interactive chat with us at https://discord.gg/Bm55Z9Ppgn.

We use github issues as our issue tracker and use github pull requests to accept edits, changes, additions and more.

If you'd like to contribute, but are not sure where to start, the list of open issues labeled with "good first issue" may give you inspiration of things to contribute. Please, also don't be shy to reach out to us on Discord.

We follow the Contributor Covenant Code of Conduct in this project.

For more details on how to write text for the book, please read contributing.md. If after reading that, you think some specific aspects could be explained better, please do let us know by raising an issue.

Index

AArch64, 36
Address Sanitizer (ASan), 35
Address space layout randomization (ASLR), 22
Application Binary Interface (ABI), 38
arbitrary code execution, 10
architectural effects, 53
arm64e, 24, 25
ASLR, 51

backward-edge CFI, 22
BlueThunder, 50
bounds check elimination, 40
Bounds check elimination (BCE), 41
branch predictor, 50
Branch Target Buffer (BTB), 51
Branch Target Identification (BTI), 32
Branch target predictor, 51
branch target predictor, 50
BranchScope, 50
BranchShadow, 51

C, 6, 7, 12, 35
C++, 6, 7, 12, 21, 24, 35
cache, 44
  access time, 45
  block, 45
  direct-mapped, 46
  eviction, 45
  fully-associative, 46
  hit, 45
  line, 45
  LRU, 46
  miss, 45
  multi-level, 45
  pseudo-LRU replacement policy, 46
  random replacement policy, 46
  replacement policy, 45
  set, 46
  set-associative, 46
  side-channel, 47
  size, 45
capability, 39
Clang, 35
coarse-grained CFI, 22
Collide+Probe, 48
Common Weakness Enumeration (CWE), 13
conditional branch direction predictor, 50
control data, 33
control dependency, 52
Control-flow integrity (CFI), 22
Coroutine Frame-Oriented Programming (CFOP), 28
Counterfeit Object-oriented programming (COOP), 21
covert channel, 43
cpu multithreading, 49

data dependency, 52
data-only attacks, 33
dead store optimization (DSO), 60
deoptimization, 40
dispatcher gadget, 20

equivalence class, 24
Evict+Reload, 48
Evict+Time, 48
exploit primitive, 7

faulting instructions, 54
fine-grained CFI, 22
Flush+Flush, 48
Flush+Prefetch, 48
Flush+Reload, 47
forward-edge CFI, 22
free, 36
function signature, 24
fuzzing, 35

gadget scanner, 18

gadgets, 16
GCC, 35
Global Offset Table(GOT), 28
GWP-ASan, 37

Hardware-assisted AddressSanitizer (HWASAN), 36
history-based prediction, 50
Hyper-Channel, 51

info leak, 22
instruction pipeline, 50
instruction trace, 51
interactive attack, 8
intra-object overflow, 38

jump table, 25
Jump-oriented programming (JOP), 20
just-in-time (JIT) compilers, 39

KASAN, 36
key, 30

last branch record, 51
LeakSanitizer, 35
locality of reference, 45

malloc, 36
measurement sequence, 48, 51
Meltdown, 54
memory access time, 45
Memory Tagging Extension (MTE), 36
memory vulnerability, 22
MemorySanitizer, 37
memset, 60
MemTagSanitizer, 36
micro-architectural, 43
micro-architectural effects, 53
mis-speculation, 53
modifier, 30

NetSpectre, 49
non-control data attacks, 33
non-interactive (one-shot) attack, 8

on-stack replacement (OSR), 40
output dependency, 52

pauthabi, 24, 25
performance counter, 51
pipeline, 50
Pointer Authentication, 29
Pointer Authentication Code (PAC), 29
pointer compression, 41
pointer substitution attack, 30
pointer substitution attacks, 32
predict, 50
prediction, 52
Prime+Probe, 47
principle of locality, 45
Procedure Linkage Table (PLT), 28

range analysis, 40
raw pointer, 30
read primitive, 8, 37
redzone, 36
register, 22
Reload+Refresh, 48
reset sequence, 48, 51
return address, 27
return address predictor, 50, 51
return-oriented programming (ROP), 16
ROP chain, 16

sanitizers, 35
SBPA, 51
shadow memory, 35
shadow stack, 29
shellcode, 12, 16
side-channel, 43
signed pointer, 30
sigreturn-oriented programming (SROP), 21
site isolation, 49
spatial memory safety, 37
Spectre, 49, 54
speculation, 52
spy, 43
stack pivoting, 17

taken branch, 50
ThreadSanitizer, 37
timing attacks, 43
Top-Byte Ignore (TBI), 36
transient execution attacks, 54
transient instructions, 53, 54
trigger sequence, 48, 51
true data dependency, 52

UndefinedBehaviorSanitizer (UBSan), 37

Use after free (UaF), 13

vfgadgets, 21
victim, 43

W^X, 42
write primitive, 7, 37
wrong-path instructions, 53

References

"10.604 Memset_explicit." n.d. https://www.gnu.org/software//gnulib/manual/html_node/memset_005fexplicit.html.

Aciiçmez, Onur, Çetin Kaya Koç, and Jean-Pierre Seifert. 2007. "On the Power of Simple Branch Prediction Analysis." In *Proceedings of the 2nd ACM Symposium on Information, Computer and Communications Security*. ACM. https://doi.org/10.1145/1229285.1266999.

Aleph One. 1996. "Smashing the Stack for Fun and Profit." 1996. http://www.phrack.org/issues/49/14.html#article.

Angelico, Chris, Tim Peters, and Guido van Rossum. 2018. "Assignment Expressions." PEP 572. https://peps.python.org/pep-0572/.

Beer, Ian. 2020. "An iOS Zero-Click Radio Proximity Exploit Odyssey." 2020. https://googleprojectzero.blogspot.com/2020/12/an-ios-zero-click-radio-proximity.html.

Beer, Ian, and Samuel Groß. 2021. "A Deep Dive into an NSO Zero-Click iMessage Exploit: Remote Code Execution." 2021. https://googleprojectzero.blogspot.com/2021/12/a-deep-dive-into-nso-zero-click.html.

Bletsch, Tyler, Xuxian Jiang, Vince W. Freeh, and Zhenkai Liang. 2011. "Jump-Oriented Programming: A New Class of Code-Reuse Attack." In *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security*, 30–40. ASIACCS '11. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/1966913.1966919.

Bosman, Erik, and Herbert Bos. 2014. "Framing Signals - a Return to Portable Shellcode." In *2014 IEEE Symposium on Security and Privacy*, 243–58. https://doi.org/10.1109/SP.2014.23.

Boucher, Nicholas, and Ross Anderson. 2023. "Trojan Source: Invisible Vulnerabilities." In *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association. https://arxiv.org/abs/2111.00169.

Briongos, Samira, Pedro Malagon, Jose M. Moya, and Thomas Eisenbarth. 2020. "RELOAD+REFRESH: Abusing Cache Replacement Policies to Perform Stealthy Cache Attacks." In *29th USENIX Security Symposium (USENIX Security 20)*. https://www.usenix.org/conference/usenixsecurity20/presentation/briongos.

Bulck, Jo Van, Daniel Moghimi, Michael Schwarz, Moritz Lippi, Marina Minkin, Daniel Genkin, Yuval Yarom, Berk Sunar, Daniel Gruss, and Frank Piessens. n.d. "LVI: Hijacking Transient Execution Through Microarchitectural Load Value Injection." In *2020 IEEE Symposium on Security and Privacy (SP)*.

Bulygin, Yuriy. 2008. "CPU Side-Channels Vs. Virtualization Rootkits: The Good, the Bad, or the Ugly." In *CPU Side-Channels Vs. Virtualization*

Rootkits: The Good, the Bad, or the Ugly. https://infocondb.org/con/toorcon/toorcon-seattle-2008/cpu-side-channels-vs-virtualization-rootkits-the-good-the-bad-or-the-ugly.

Burow, Nathan, Scott A. Carr, Joseph Nash, Per Larsen, Michael Franz, Stefan Brunthaler, and Mathias Payer. 2017. "Control-Flow Integrity: Precision, Security, and Performance." *ACM Comput. Surv.* 50 (1). https://doi.org/10.1145/3054924.

Cheeseman, Luke. 2019. "Code Reuse Attacks: The Compiler Story." 2019. https://community.arm.com/arm-community-blogs/b/tools-software-ides-blog/posts/code-reuse-attacks-the-compiler-story.

Chen, Shuo, Jun Xu, Emre Can Sezer, Prachi Gauriar, and Ravishankar K Iyer. 2005. "Non-Control-Data Attacks Are Realistic Threats." In *USENIX Security Symposium*, 5:146.

Conti, Mauro, Stephen Crane, Lucas Davi, Michael Franz, Per Larsen, Marco Negro, Christopher Liebchen, Mohaned Qunaibit, and Ahmad-Reza Sadeghi. 2015. "Losing Control: On the Effectiveness of Control-Flow Integrity Under Stack Attacks." In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 952–63. CCS '15. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/2810103.2813671.

Cook, Kees. 2023. "Bounded Flexible Arrays in C." 2023. https://people.kernel.org/kees/bounded-flexible-arrays-in-c.

Cox, Joseph. 2015. "Hack Brief: Malware Sneaks into the Chinese iOS App Store." *WIRED*. https://www.wired.com/2015/09/hack-brief-malware-sneaks-chinese-ios-app-store/.

D'Silva, Vijay, Mathias Payer, and Dawn Song. 2015. "The Correctness-Security Gap in Compiler Optimization." In *2015 IEEE Security and Privacy Workshops*, 73–87. IEEE.

Denis-Courmont, R., H. Liljestrand, C. Chinea, and J. -E. Ekberg. 2021. "Camouflage: Hardware-Assisted CFI for the ARM Linux Kernel." In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. https://doi.org/https://doi.org/10.1109/DAC18072.2020.9218535.

Dowd, Mark, John McDonald, and Justin Schuh. 2006. *The Art of Software Security Assessment: Identifying and Preventing Software Vulnerabilities*. Addison-Wesley Professional.

Du, Jianhao Xu1 Kangjie Lu2 Zhengjie, Zhu Ding1 Linke Li1 Qiushi Wu, and Mathias Payer3 Bing Mao. n.d. "Silent Bugs Matter: A Study of Compiler-Introduced Security Bugs."

Dullien, Thomas. 2020. "Weird Machines, Exploitability, and Provable Unexploitability." *IEEE Transactions on Emerging Topics in Computing* 8 (2): 391–403. https://doi.org/10.1109/TETC.2017.2785299.

Evtyushkin, Dmitry, Ryan Riley, Nael CSE, ECE Abu-Ghazaleh, and Dmitry Ponomarev. 2018. "BranchScope: A New Side-Channel Attack on Directional Branch Predictor." In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*. https://doi.org/10.1145/3173162.3173204.

Eyerman, Stijn, Lieven Eeckhout, Tejas Karkhanis, and James E. Smith. 2009. "A Mechanistic Performance Model for Superscalar Out-of-Order Processors." *ACM Transactions on Computer Systems* 27 (2). https://doi.org/10.1145/1534909.1534910.

farkhani, Reza Mirzazade, Mansour Ahmadi, and Long Lu. 2021. "PTAuth: Temporal Memory Safety via Robust Points-to Authentication." In *30th USENIX Security Symposium (USENIX Security 21)*. https://www.usenix.org/conference/usenixsecurity21/presentation/mirzazade.

Fetiveau, Jeremy. 2019. "Circumventing Chrome's Hardening of Typer Bugs." 2019. https://doar-e.github.io/blog/2019/05/09/circumventing-chromes-hardening-of-typer-bugs/.

Flake, Thomas Dullien/Halvar. 2018. "Turing Completeness, Weird Machines, Twitter, and Muddled Terminology." 2018. http://addxorrol.blogspot.com/2018/10/turing-completeness-weird-machines.html.

"FreeBSD Mitigation for Srandom Undefined Behavior." 2012. 2012. https://github.com/freebsd/freebsd-src/commit/6a762eb23ea5f31e65cfa12602937f39a14e9b0c.

"GCC6 - TB Crashes Due to Removed Null Pointer Checks for "This"." 2016. 2016. https://bugzilla.mozilla.org/show_bug.cgi?id=1251576.

Glazunov, Sergei. 2021. "In-the-Wild Series: Chrome Infinity Bug." 2021. https://googleprojectzero.blogspot.com/2021/01/in-wild-series-chrome-infinity-bug.html.

Gostev, Alexander. 2009. "A Short History of Induc." 2009. https://securelist.com/a-short-history-of-induc/30555/.

Greenberg, Andy. 2019. "Supply Chain Hackers Snuck Malware into Videogames." *WIRED*. https://www.wired.com/story/supply-chain-hackers-videogames-asus-ccleaner/.

Groß, Samuel. 2020. "JITSploitation i: A JIT Bug." 2020. https://googleprojectzero.blogspot.com/2020/09/jitsploitation-one.html.

Groß, Samuel, and Amy Burnett. 2022. "Attacking JavaScript Engines in 2022." 2022. https://saelo.github.io/presentations/offensivecon_22_attacking_javascript_engines.pdf.

Gruss, Daniel, Clémentine Maurice, Anders Fogh, Moritz Lipp, and Stefan Mangard. 2016. "Prefetch Side-Channel Attacks: Bypassing SMAP and Kernel ASLR." In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. CCS '16. https://doi.org/10.1145/2976749.2978356.

Gruss, Daniel, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. 2016. "Flush+flush: A Fast and Stealthy Cache Attack." In *Proceedings of the 13th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment - Volume 9721*, 279–99. DIMVA 2016. https://doi.org/10.1007/978-3-319-40667-1_14.

Hicks, Michael. 2014. "What Is Memory Safety?" 2014. http://www.pl-enthusiast.net/2014/07/21/memory-safety/.

Hu, Hong, Shweta Shinde, Sendroiu Adrian, Zheng Leong Chua, Prateek Saxena, and Zhenkai Liang. 2016. "Data-Oriented Programming: On the Expressiveness of Non-Control Data Attacks." In *2016 IEEE Symposium on Security and Privacy (SP)*, 969–86. https://doi.org/10.1109/SP.2016.62.

Huo, Tianlin, Xiaoni Meng, Wenhao Wang, Chunliang Hao, Pei Zhao, Jian Zhai, and Mingshu Li. 2019. "Bluethunder: A 2-Level Directional Predictor Based Side-Channel Attack Against SGX." *IACR Transactions on Cryptographic Hardware and Embedded Systems*, November. https://doi.org/10.46586/tches.v2020.i1.321-347.

Ispoglou, Kyriakos K., Bader AlBassam, Trent Jaeger, and Mathias Payer.

2018. "Block Oriented Programming: Automating Data-Only Attacks." In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 1868–82. CCS '18. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3243734.3243739.

"Issue 3782: V8 Is Not -Fsanitize=null Clean." 2014. 2014. https://bugs.chromium.org/p/v8/issues/detail?id=3782.

Kapil, Dhaval. 2022. "Heap Exploitation." 2022. https://heap-exploitation.dhavalkapil.com/attacks.

Kocher, Paul, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, et al. 2019. "Spectre Attacks: Exploiting Speculative Execution." In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE. https://doi.org/10.1109/sp.2019.00002.

Korobeynikov, Anton. 2024. "Adding Pointer Authentication ABI Support for Your ELF Platform." 2024. https://llvm.org/devmtg/2024-10/slides/techtalk/Korobeynikov-Adding-Pointer-Authentication-ABI-support.pdf.

Lee, Sangho, Ming-Wei Shih, Prasun Gera, Taesoo Kim, Hyesoon Kim, and Marcus Peinado. 2017. "Inferring Fine-Grained Control Flow Inside SGX Enclaves with Branch Shadowing." In *26th USENIX Security Symposium (USENIX Security 17)*. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/lee-sangho.

"Lib: Make Memzero_explicit More Robust Against Dead Store Elimination." 2015. 2015. https://github.com/torvalds/linux/commit/7829fb09a2b4268b30dd9bc782fa5ebee278b137.

———. 2016. 2016. https://github.com/openssl/openssl/commit/104ce8a9f02d250dd43c255eb7b8747e81b29422.

"Lib: Memzero_explicit: Use Barrier Instead of OPTIMIZER_HIDE_VAR." 2015. 2015. https://github.com/torvalds/linux/commit/0b053c9518292705736329a8fe20ef4686ffc8e9.

Liljestrand, Hans, Thomas Nyman, Lachlan J Gunn, Jan-Erik Ekberg, and N Asokan. 2021. "{PACStack}: An Authenticated Call Stack." In *30th USENIX Security Symposium (USENIX Security 21)*, 357–74.

Liljestrand, Hans, Thomas Nyman, Kui Wang, Carlos Chinea Perez, Jan-Erik Ekberg, and N. Asokan. 2019. "PAC It up: Towards Pointer Integrity Using ARM Pointer Authentication." In *Proceedings of the 28th USENIX Conference on Security Symposium*.

Lipp, Moritz, Vedad Hadzic, Michael Schwarz, Arthur Perais, Clementine Lucie Noemie Maurice, and Daniel Gruß. 2020. "Take a Way: Exploring the Security Implications of AMD's Cache Way Predictors." In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security, ASIA CCS 2020*. https://doi.org/10.1145/3320269.3384746.

Lipp, Moritz, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, et al. 2018. "Meltdown: Reading Kernel Memory from User Space." In *27th USENIX Security Symposium (USENIX Security 18)*. https://www.usenix.org/conference/usenixsecurity18/presentation/lipp.

McCall, John, and Ahmed Bougacha. 2019. "Arm64e: An ABI for Pointer Authentication." 2019. https://llvm.org/devmtg/2019-10/slides/McCall-Bougacha-arm64e.pdf.

"Memset_explicit." 2021. 2021. https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2897.htm.

Miller, Matt. n.d. "Modeling the Exploitation and Mitigation of Memory Safety

Vulnerabilities." Breakpoint 2012. https://github.com/Microsoft/MSRC-Security-Research/blob/master/presentations/2012_10_Breakpoint/BreakPoint2012_Miller_Modeling_the_exploitation_and_mitigation_of_memory_safety_vulnerabilities.pdf.

Mushtaq, Maria, Muhammad Asim Mukhtar, Vianney Lapotre, Muhammad Khurram Bhatti, and Guy Gogniat. 2020. "Winter Is Here! A Decade of Cache-Based Side-Channel Attacks, Detection & Mitigation for RSA." *Information Systems*, September. https://doi.org/10.1016/j.is.2020.101524.

Mutlu, Onur, Hyesoon Kim, David N. Armstrong, and Yale N. Patt. 2004. "Understanding the Effects of Wrong-Path Memory References on Processor Performance." In *Proceedings of the 3rd Workshop on Memory Performance Issues in Conjunction with the 31st International Symposium on Computer Architecture - WMPI '04*. https://doi.org/10.1145/1054943.1054951.

"OpenBSD Mitigation for Srandom Undefined Behavior." 2002. 2002. https://github.com/openbsd/src/commit/99d815f892ce481695caf21f08f773f563820a66.

Osvik, Dag Arne, Adi Shamir, and Eran Tromer. 2005. "Cache Attacks and Countermeasures: The Case of AES." In *IACR Cryptology ePrint Archive*. https://doi.org/10.1007/11605805_1.

Paul A., Karger, and Schell Roger R. 1974. "MULTICS SECURITY EVALUATION: VULNERABILITY ANALYSIS," 52. https://csrc.nist.gov/csrc/media/publications/conference-paper/1998/10/08/proceedings-of-the-21st-nissc-1998/documents/early-cs-papers/karg74.pdf.

Percival, Colin. 2005. "Cache Missing for Fun and Profit." BSDCan. https://eprint.iacr.org/2005/271.

Pizlo, Filip. 2020. "Speculation in JavaScriptCore." 2020. https://webkit.org/blog/10308/speculation-in-javascriptcore/.

Pornin, Thomas. 2018. "Why Constant-Time Crypto?" 2018. https://www.bearssl.org/constanttime.html.

Rutland, Mark. 2017. "ARMv8.3 Pointer Authentication." 2017. https://events.static.linuxfound.org/sites/events/files/slides/slides_23.pdf.

saelo. 2021a. "Attacking JavaScript Engines: A Case Study of JavaScriptCore and CVE-2016-4622." 2021. http://www.phrack.org/issues/70/3.html.

———. 2021b. "Exploiting Logic Bugs in JavaScript JIT Engines." 2021. http://www.phrack.org/issues/70/9.html.

Schilling, Robert, Pascal Nasahl, and Stefan Mangard. 2022. "FIPAC: Thwarting Fault- and Software-Induced Control-Flow Attacks with ARM Pointer Authentication." In *Constructive Side-Channel Analysis and Secure Design: 13th International Workshop, COSADE 2022, Leuven, Belgium, April 11-12, 2022, Proceedings*. https://doi.org/10.1007/978-3-030-99766-3_5.

Schuster, F., T. Tendyck, C. Liebchen, L. Davi, A. Sadeghi, and T. Holz. 2015. "Counterfeit Object-Oriented Programming: On the Difficulty of Preventing Code Reuse Attacks in c++ Applications." In *2015 IEEE Symposium on Security and Privacy (SP)*, 745–62. Los Alamitos, CA, USA: IEEE Computer Society. https://doi.org/10.1109/SP.2015.51.

Schwarz, Michael, Clémentine Maurice, Daniel Gruss, and Stefan Mangard. 2017. "Fantastic Timers and Where to Find Them: High-Resolution Microarchitectural Attacks in JavaScript." In *Financial Cryptography and Data Security*, 247–67. Springer International Publishing.

Schwarz, Michael, Martin Schwarzl, Moritz Lipp, Jon Masters, and Daniel

Gruss. 2019. "NetSpectre: Read Arbitrary Memory over Network." In *Computer Security - ESORICS 2019 - 24th European Symposium on Research in Computer Security, Luxembourg, September 23-27, 2019, Proceedings, Part I*, 11735:279–99. Lecture Notes in Computer Science. Springer.

Seacord, Robert C. 2013. *Secure Coding in c and c++*. 2nd ed. Addison-Wesley Professional.

Serebryany, Konstantin, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov. 2012. "{AddressSanitizer}: A Fast Address Sanity Checker." In *2012 USENIX Annual Technical Conference (USENIX ATC 12)*, 309–18.

Serna, Fermin J. 2012. "The Info Leak Era on Software Exploitation." 2012. https://www.youtube.com/watch?v=VgWoPa8Whmc.

Shacham, Hovav. 2007. "The Geometry of Innocent Flesh on the Bone: Return-into-Libc Without Function Calls (on the X86)." In *Proceedings of the 14th ACM Conference on Computer and Communications Security*, 552–61. CCS '07. New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/1315245.1315313.

Sharma, Siddharth. 2014. "Enhance Application Security with FORTIFY_SOURCE." 2014. https://www.redhat.com/en/blog/enhance-application-security-fortifysource.

Soares, Luigi, and Fernando Magno Quintan Pereira. 2021. "Memory-Safe Elimination of Side Channels." In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE. https://doi.org/10.1109/cgo51591.2021.9370305.

Solar Designer. 1997. "Getting Around Non-Executable Stack (and Fix)." 1997. https://seclists.org/bugtraq/1997/Aug/63.

Song, Chengyu, Chao Zhang, Tielei Wang, Wenke Lee, and David Melski. 2015. "Exploiting and Protecting Dynamic Code Generation." In *NDSS*.

Su, Chao, and Qingkai Zeng. 2021. "Survey of CPU Cache-Based Side-Channel Attacks: Systematic Analysis, Security Models, and Countermeasures." *Security and Communication Networks*, June. https://doi.org/10.1155/2021/5559552.

Thompson, Ken. 1984. "Reflections on Trusting Trust." https://www.cs.cmu.edu/~rdriley/487/papers/Thompson_1984_ReflectionsonTrustingTrust.pdf.

Wang, Xi. 2015. "More Randomness or Less." 2015. https://kqueue.org/blog/2012/06/25/more-randomness-or-less/.

Wang, Xi, Haogang Chen, Alvin Cheung, Zhihao Jia, Nickolai Zeldovich, and M Frans Kaashoek. 2012. "Undefined Behavior: What Happened to My Code?" In *Proceedings of the Asia-Pacific Workshop on Systems*, 1–7.

Weber, Daniel, Ahmad Ibrahim, Hamed Nemati, Michael Schwarz, and Christian Rossow. 2021. "Osiris: Automated Discovery of Microarchitectural Side Channels." In *USENIX Security'21*. https://arxiv.org/abs/2106.03470.

Wheeler, David A. 2020. "Initial Analysis of Underhanded Source Code." Institute for Defense Analyses. https://www.ida.org/research-and-publications/publications/all/i/in/initial-analysis-of-underhanded-source-code.

Wingo, Any. 2011. "Value Representation in JavaScript Implementations." 2011. https://wingolog.org/archives/2011/05/18/value-representation-in-javascript-implementations.

Wu, Meng, Shengjian Guo, Patrick Schaumont, and Chao Wang. 2018. "Eliminating Timing Side-Channel Leaks Using Program Repair." In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing*

and Analysis. ACM. https://doi.org/10.1145/3213846.3213851.

Yarom, Yuval, and Katrina Falkner. 2014. "FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack." In *23rd USENIX Security Symposium (USENIX Security 14)*, 719–32. San Diego, CA: USENIX Association. https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/yarom.

Yoo, Sungbae, Jinbum Park, Seolheui Kim, Yeji Kim, and Taesoo Kim. 2021. "In-Kernel Control-Flow Integrity on Commodity OSes Using ARM Pointer Authentication." *CoRR*. https://arxiv.org/abs/2112.07213.